

Blockchain-Based Energy Metering and Billing: Improving Scalability, Security, and Efficiency

Tasmiya Mujawar
mtasmiya44@gmail.com

Abstract—This thesis explores the potential of blockchain technology, specifically Ethereum, in addressing inefficiencies in the energy market. The research focuses on enhancing scalability and security of smart contracts, crucial components for decentralized applications (DApps) in the energy sector. Three key research topics are investigated: improving scalability of smart contracts, enhancing smart contract security, and modeling a decentralized energy metering and billing system on the Ethereum blockchain. The findings contribute to the development of efficient and secure smart contracts, with a proof of concept implementation for energy metering and billing using Solidity. The study emphasizes the importance of industry best practices and evaluates automated security testing tools to ensure the robustness of smart contracts in real-world applications.

I. INTRODUCTION

In 2009, Satoshi Nakamoto introduced Bitcoin [1], a peer-to-peer electronic cash system, which utilizes blockchain technology for decentralized transactions. Ethereum [2] expanded on this by enabling smart contracts—self-executing contracts with terms directly written into code [3]. These innovations led to the development of Decentralized Applications (DApps) with use cases such as crowdfunding, decentralized exchanges, and decentralized autonomous organizations. However, blockchain technology faces challenges in scalability, security, and transaction throughput.

This thesis explores three main topics: improving the scalability of Ethereum’s smart contracts, enhancing smart contract security and streamlining auditing processes, and designing a blockchain-based energy metering and billing system using Ethereum, focusing on transparency, decentralization, and scalability. The energy metering system is based on a use case provided by Honda Research and Development Europe Germany (HREG).

The work focuses on key DApp quality aspects such as scalability and security, applied to a proof of concept for energy metering and billing with Solidity smart contracts. Security practices follow industry standards, and an automated security testing tool is evaluated. Technologies used include Solidity, the Truffle Framework, JavaScript for testing, and Python for automation and data analysis.

II. ETHEREUM AND BLOCKCHAIN BASICS

A blockchain is a growing list of *blocks* containing metadata and transactions, linked by *hashes*. As new blocks are added, older blocks become more secure [4].

A *cryptographic hash function* maps data to a fixed-size string with properties like collision and pre-image resistance [5]. Ethereum uses KECCAK-256 [6], [7].

Public Key Cryptography uses paired keys for secure communication, ensuring confidentiality, authentication, and integrity through encryption and digital signatures.

Blockchains are distributed, immutable ledgers that allow trustless transactions, offering transparency and censorship resistance, but with privacy concerns. Privacy-enabled blockchains like Monero and Zcash address these [8], [9].

Blockchains are either *Public/Permissionless* (transparent, immutable) or *Private/Permissioned* (restricted access, privacy-focused) [10]. Corporations use permissioned blockchains for scalability and privacy [11], [12], [13].

III. ETHEREUM BLOCKCHAIN

The Ethereum blockchain acts as a transactional state machine. The first state is the *genesis* state referred to as the *genesis block*. After the execution of each transaction, the state changes. Transactions are collated together in *blocks*.

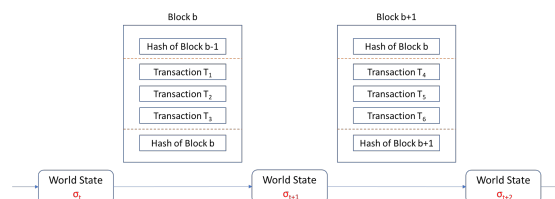


Fig. 1: Ethereum can be seen as a chain of states [14]

A valid state transition requires the appending of a new block to the existing list of blocks. Each block contains transactions and a reference to the previous block, forming a chain. In Ethereum, the only way for a block to be validated and appended to the list is through a validation process called mining. Mining involves a group of computers, known as miners, expending their computational resources to find the solution to a puzzle. The first miner to find a solution to the puzzle is rewarded with Ether¹ and is able to validate their block proposal. This is a process known as *Proof of Work* (PoW) [15].

Due to a large number of miners competing to solve the PoW puzzle, sometimes a miner might solve the PoW at the same time as another miner but for different block contents. This results in a *fork* of the blockchain. Nodes will accept the first valid block that they receive². Each blockchain protocol

¹The Ethereum network’s native currency

²This depends on block propagation time based on bandwidth, block-size, connectivity, etc.

has a way to resolve forks and determine which chain is the valid chain. In Ethereum, the longest chain is based on total difficulty³ which can be found in the *blockheader*. Ethereum is advertised to be using a modification of the GHOST Protocol [16] as its chain selection mechanism, which uses *uncle blocks*⁴. This is contradictory since Ethereum's uncle blocks do not count towards difficulty, and as a result, Ethereum does not actually use an adaptation of the GHOST protocol [17]; the uncle reward is just used to reduce miner centralization.

IV. ETHEREUM VIRTUAL MACHINE

The Ethereum Virtual Machine (EVM) is a Turing-complete runtime environment that allows the execution of complex computations on the Ethereum blockchain. It supports smart contracts, which are self-executing agreements that run on the blockchain. Ethereum accounts are classified into two types: Externally Owned Accounts (EOAs) and Contract Accounts (CAs). Each account holds specific state information, including a nonce (a counter for transactions), a balance, storage hash, and code hash. The Ethereum world state is a mapping from addresses to account states, and it uses Merkle Trees to allow efficient verification of data integrity.

Transactions in Ethereum are initiated by EOAs, which sign and send them. A transaction consists of several fields: gas, nonce, input data, and a cryptographic signature. Gas is a unit of computation that limits the work a transaction can perform. It is used to pay for every operation that the transaction executes. The gasPrice is specified by the sender and determines the cost of gas, influencing the transaction's priority. Miners prioritize transactions with higher gas prices, as these offer more incentives.

Each block in the Ethereum blockchain contains a header, transaction hashes, and other metadata, such as the gas used, miner address, and receipts root. Gas is a crucial component of Ethereum's transaction mechanism. The gasLimit of a block specifies the maximum amount of gas that can be used for all transactions within that block. Miners can adjust the gasLimit depending on network conditions, but it is typically set by the protocol. The transaction cost is determined by multiplying the gasPrice by the gasUsed, and miners are incentivized to include transactions with higher fees, particularly when the network experiences congestion.

V. PROGRAMMING IN ETHEREUM

The Ethereum Virtual Machine (EVM) utilizes its own Turing-complete bytecode, with more significant level dialects like Robustness, Snake, LLL, and Vyper ordered into EVM code. Robustness, frequently contrasted with JavaScript, is the most famous language and has an article situated plan looking like C++ or Java. The Strength Compiler (`solc`) is utilized

to produce the EVM bytecode and the Application Double Connection point (ABI) for deployment [20], [21].

A. Tooling

1) *Client Executions and Testnets*: Ethereum's primary client executions are Geth (Golang) and cpp-ethereum (C++), with outsider options like Equality (Rust), Pyethereum (Python), and Ethereum (Java). Geth upholds the Rinkeby testnet, and Equality upholds Kovan. Public testnets like Kovan, Rinkeby (Confirmation of Power), and Ropsten (Verification of Work) consider free testing. Designers can likewise set up nearby testnets utilizing Geth or Equality, or use devices like Ganache for private blockchain recreations.

2) *Web3*: Web3.js is the essential library for communicating with Ethereum hubs and building decentralized applications (DApps). Different executions, as Web3.py, are likewise accessible. Web3.js and Web3.py permit engineers to interface with Ethereum networks like Ropsten and Mainnet, with full documentation accessible online [?], [?].

3) *Truffle Framework*: The Truffle Structure is a NodeJS-based improvement system for savvy contracts. It robotizes contract organization through movement scripts, incorporates testing with Mocha, and gives a debugger to exchange execution. It additionally incorporates with Ganache for neighborhood testing.

VI. RELATED WORK

Strategies for additional proficient shrewd agreements, for example, putting away information off-chain in exchange inputs, are portrayed in [22]. This technique lessens the requirement for shrewd agreements and considers handling through applications by checking exchanges.

On network-level adaptability, different methodologies like off-chain exchanges, sidechains, and permissioned blockchains are explored [23], [24], [25], [26], [27], [28], [?], [29], [30], [12], [31].

Cheng et al. feature that savvy contract compiler improvements are expected to diminish gas costs [32]. They created GASPER, a device that distinguishes expensive examples in Ethereum savvy contracts.

Security investigations of Ethereum savvy contracts have distinguished weaknesses, with a lot of assets being stolen [17], [33], [34], [35], [36]. Apparatuses have been created to distinguish weaknesses in both source code and bytecode [37], [38], [39], [40], [41], [42].

Blockchain applications in the Web of Things (IoT) and energy markets are being explored [43], [44], [45], [46], [47], [48], [49].

VII. BLOCKCHAIN SCALABILITY

Versatility is critical for blockchain networks. Ethereum, for example, can add a block like clockwork yet is restricted by the block size and gas limits [50]. Exchange throughput is limited by these elements.

³Difficulty is a measure of how much computational effort needs to be performed by a miner to solve a PoW puzzle. Total Difficulty is the sum of the difficulties of all blocks until the examined block.

⁴In Bitcoin, a block with a valid PoW that arrived to a node after another valid block at the same height is called an orphan because it gets discarded by Bitcoin's algorithm. In Ethereum, these blocks do not get discarded; instead, they are added to the chain as *uncle blocks* and receive a reduced block reward

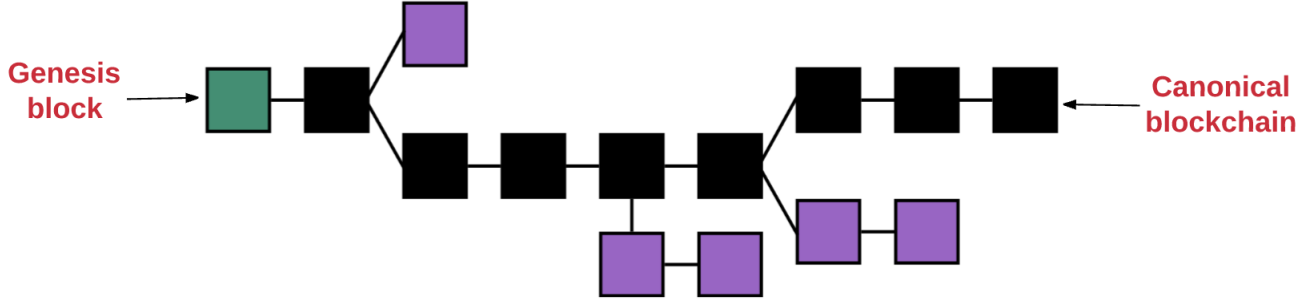


Fig. 2: The Ethereum protocol chooses the longest chain [18]

VIII. BOTTLENECKS IN SCALABILITY

Versatility can be addressed by upgrading contract plan to diminish gas costs, permitting more exchanges per block. In any case, Ethereum's blockGasLimit limits the throughput, making it less cutthroat contrasted with conventional installment frameworks.

IX. NETWORK LEVEL SCALABILITY

Blockchain versatility isn't just about expanding exchange throughput yet adjusting decentralization, adaptability, and security. This is the Versatility Trilemma [51].

X. PROOF OF STAKE

Verification of Stake (PoS) replaces diggers with validators who stake ether to approve blocks, offering diminished energy utilization and quicker block times. Ethereum is changing to PoS with the Casper protocol [52].

XI. SIDECHAINS

Sidechains permit offloading calculations from the primary blockchain, improving for explicit applications while depending on the mainchain for security. Ethereum's Plasma arrangement utilizes sidechains for scalability [30].

XII. SHARDING

Sharding parts the blockchain's interaction across different hubs, each dealing with a subset of exchanges. It is a complicated arrangement still under research [51].

XIII. STATE CHANNELS

State diverts work with off-chain exchanges, with blockchain records possibly made when the cycle finishes up. This strategy is especially helpful for miniature transactions [?].

XIV. CONTRACT LEVEL SCALABILITY

An investigation of 4240 brilliant agreements found that more than 70% cause superfluous gas costs because of compiler inefficiencies [32]. The Strength compiler frequently neglects to advance struct types, prompting repetitive operations [54]. In this part, I investigate how gas is registered and the way that savvy agreements can be upgraded to lessen exchange charges.

TABLE I: Gas costs for different operations [32]

Operation	Gas	Description
ADD/SUB	3	Arithmetic operation
MUL/DIV	5	
ADDMOD/MULMOD	8	
AND/OR/XOR	3	Bitwise logic operation
LT/GT/SLT/SGT/EQ	3	Comparison operation
POP	2	Stack operation
PUSH/DUP/SWAP	3	
MLOAD/MSTORE	3	
JUMP	8	Unconditional jump
JUMPI	10	Conditional jump
SLOAD	200	Storage operation
SSTORE	5,000/20,000	
BALANCE	400	
CREATE	32,000	Create a new account using CREATE
CALL	25,000	Create a new account using CALL

A. Gas Costs

An Ethereum exchange's absolute gas cost incorporates Base Exchange Expenses and Execution Expenses. The Base Exchange Costs cover fixed costs like sending information and conveying contracts, while Execution Expenses allude to the computational expense of agreement operations [4], [55]. Key activities like Make, CALL, and SSTORE bring about high gas costs. Limiting these activities can decrease charges and further develop versatility.

To decrease gas costs, it is crucial for use compiler advancements, reuse code through libraries [56], and erase unused factors to recover gas. Staying away from huge information stockpiling on the blockchain by utilizing hashes rather is likewise suggested. Robustness' forthcoming updates expect to address some inefficiencies [54].

B. Gas Investment funds Case Study

I look at three techniques for putting away information in a savvy contract for a gaming use case. These strategies incorporate pressing qualities utilizing Strength's streamlining agent, physically pressing characteristics into uint256 factors, and pressing attributes into bytes32 factors utilizing bitwise tasks and Robustness Libraries. The strategies' gas costs are looked at in Addendum F.

1) *Using Strength's Streamlining agent and struct Variables:* This technique bunches character qualities into a struct, making the code more decipherable and permitting

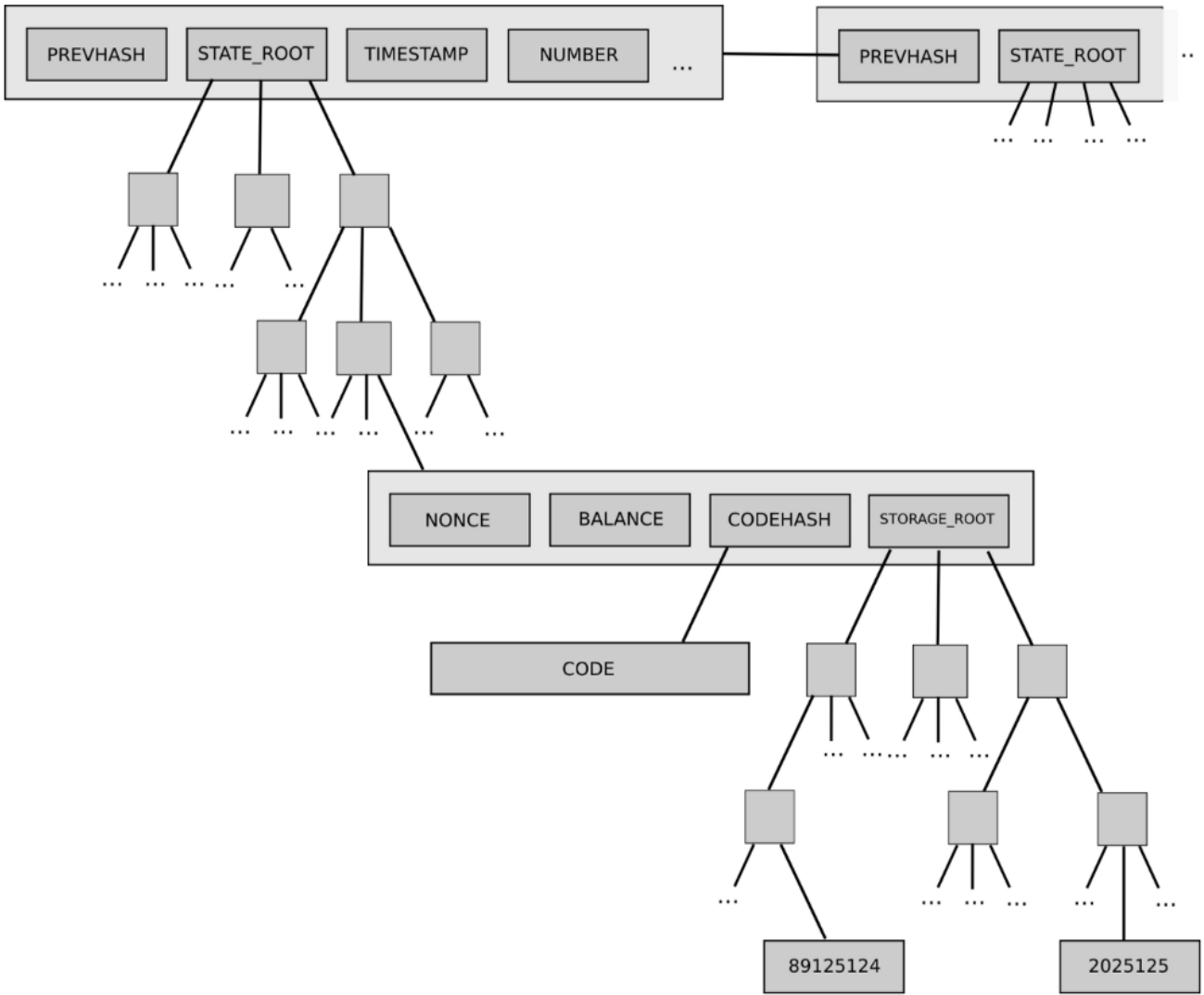


Fig. 3: The Ethereum world state [2]

Robustness' enhancer to oversee gas utilization. In any case, this strategy may not be as gas-efficient as others for bigger agreements.

2) *Manually Pressing Qualities into uint256 Variables:* In this strategy, characteristics are physically encoded into a solitary uint256 variable utilizing bitwise tasks. This approach is a larger number of gas-efficient than utilizing structs, diminishing agreement bytecode size and further developing capability call effectiveness.

3) *Using bytes32 Factors with Strength Libraries:* This strategy packs attributes into bytes32 factors utilizing bitwise tasks, with the additional advantage of Strength Libraries for code reuse and viability. While more proficient than the past strategies, it very well might be less lucid.

C. Results

Table III, IV, and V show that the manual pressing technique (uint256) is the most gas-efficient, offering a 29%

improvement in sending and a 36% improvement in capability brings over the struct-based strategy. Nonetheless, for huge scope contracts requiring practicality and intelligibility, the bytes32 technique with libraries is ideal.

XV. PAST ATTACKS

A. Network Level Attacks

a) *October 2016 Spam Attacks:* In September-October 2016, an aggressor overflowed the Ethereum network by making 19 million 'dead' accounts involving a weakness in the Self destruction opcode. This prompted a Refusal of Administration assault, creating setbacks for client synchronization. The organization answered with two hard-forks: Tangerine Whistle and Deceptive Winged serpent, which fixed the issue and cleared the records [57].

b) *Eclipse Assaults on Ethereum:* Obscure assaults, where an aggressor floods a casualty hub with malevolent

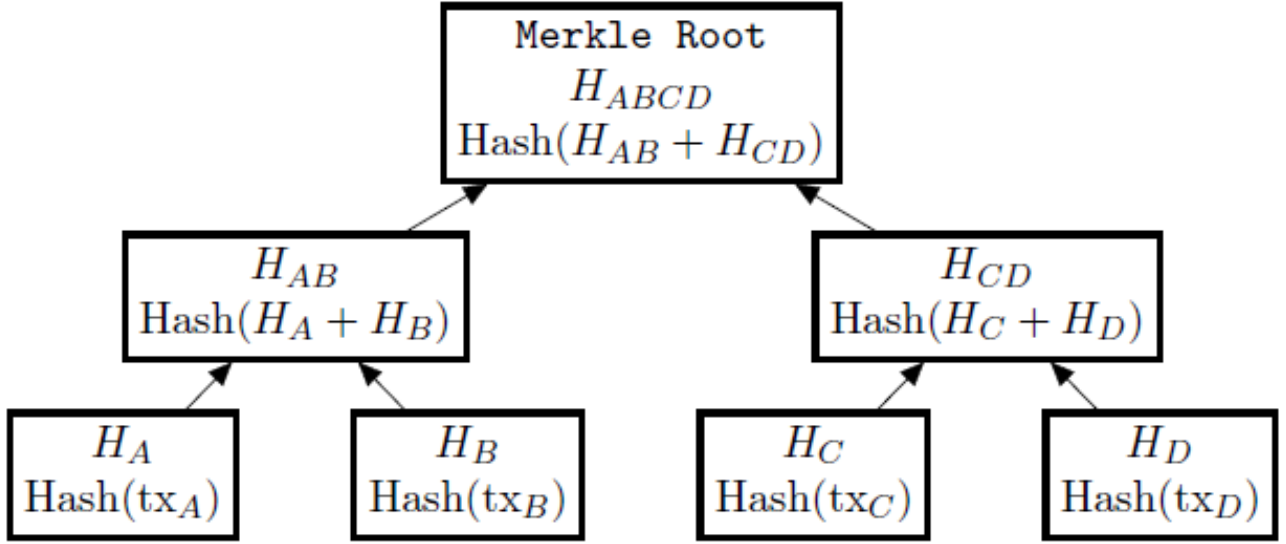


Fig. 4: Node calculation in a Merkle Tree [19]

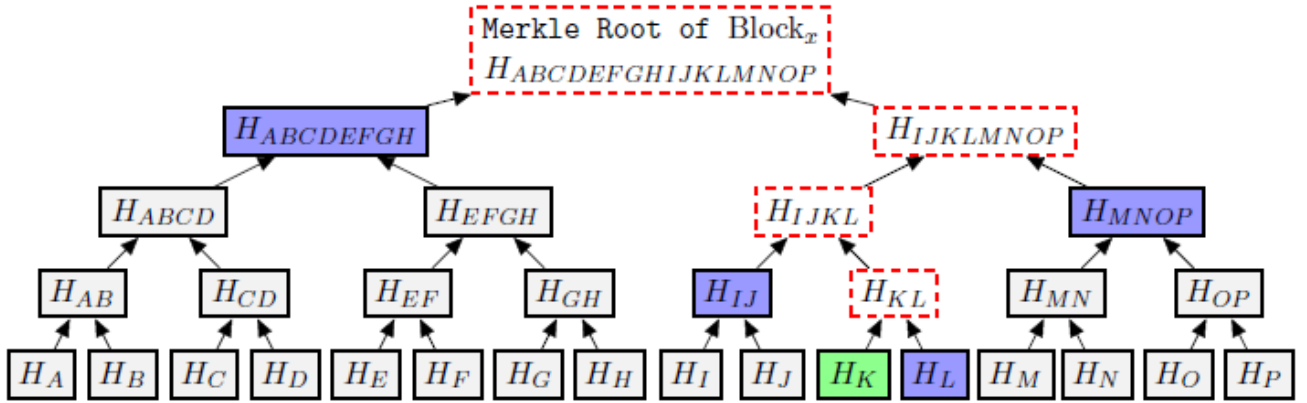


Fig. 5: Merkle path calculation [19]

associations, were found to influence Ethereum because of its organized organization convention. This weakness, which could cause twofold spending, was fixed in the Geth client in adaptation 1.8 [58]. Different clients like Equality were unaffected, showing the significance of assorted convention executions.

B. Smart Agreement Attacks

1) *TheDAO*: TheDAO, a decentralized business store, was taken advantage of in 2016 through a reentrancy assault on the savvy contract, permitting the burglary of 3.6 million ether (\$50 million). The assault prompted a disputable hard-fork, making Ethereum and Ethereum Exemplary [59], [36].

2) *Parity Multisig 1*: In 2017, a weakness in Equality's multisig wallet permitted an aggressor to take 150,000 ether by

taking advantage of an unprotected `initWallet` capability. This was alleviated by moral programmers who saved \$85 million worth of ether [34], [60].

3) *Parity Multisig 2*: After the principal assault, Equality conveyed a new multisig wallet with a safeguarded `initWallet` capability. In any case, a uninitialized contract permitted the aggressor to reinitialize and erase the library, freezing 513,774 ether. Recommendations to recuperate the assets were at last excused [35], [61], [62].

XVI. EVALUATING SAVVY AGREEMENT SECURITY

Because of the great monetary stakes, savvy contracts require careful security reviews, bug-bounties, and the most recent variant of devices like the Robustness Compiler. Notwithstanding, weaknesses frequently arise quicker than existing

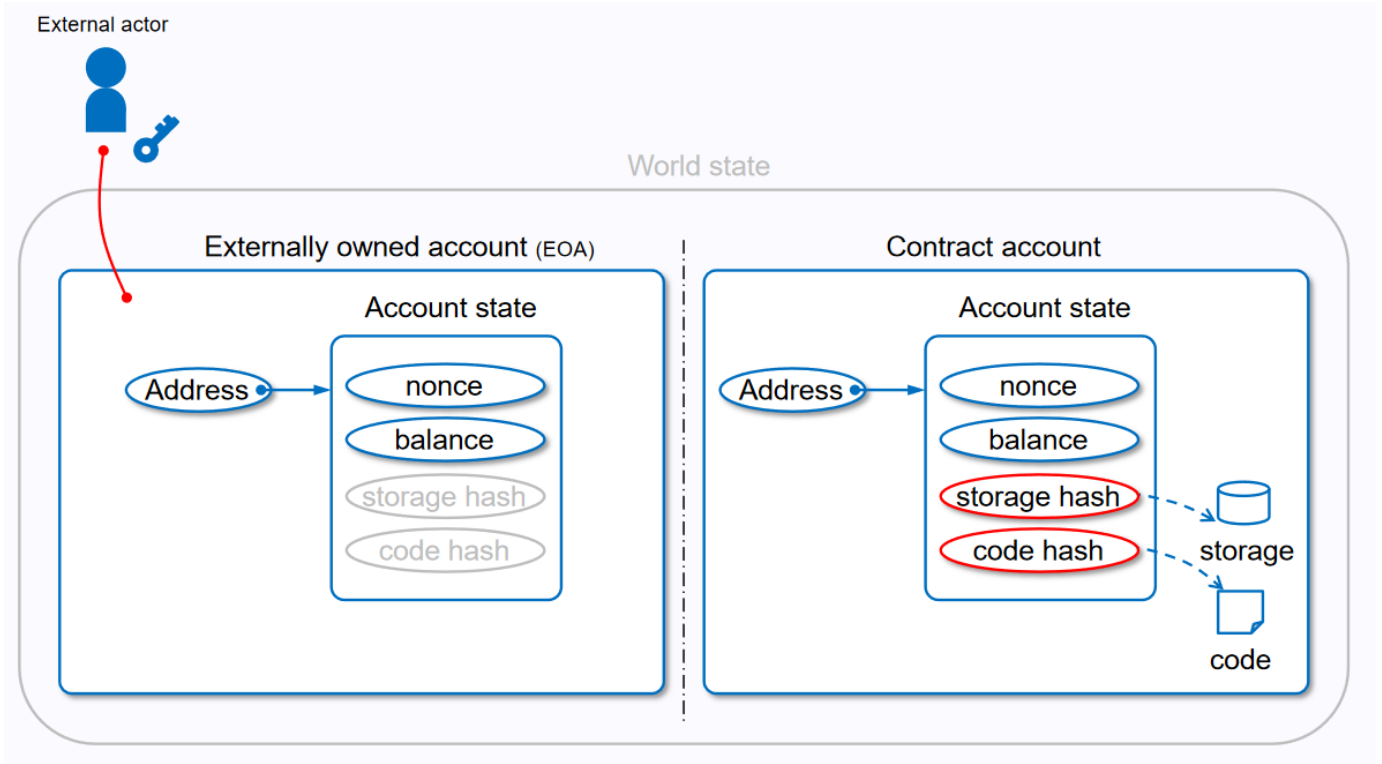


Fig. 6: Externally Owned Accounts and Contract Accounts in the Ethereum World state [14]

TABLE II: Required variables and sizes to describe a Character.

Name	Type	Comment
playerID	uint16	Game supports up to 65535 players
creationTime	uint32	Game supports timestamps up to $2^{32} = 02/07/2106$ @ 6:28am (UTC)
class	uint4	Game supports up to 16 classes
race	uint4	Game supports up to 16 classes
strength	uint16	Stats can be up to 65535
agility	uint16	Stats can be up to 65535
wisdom	uint16	Stats can be up to 65535
metadata	bytes18	Utilize the rest of the word for metadata

TABLE III: Gas costs for deployment and for each function using Solidity's built-in structs

Optimizer Runs	Register	CreateCharacter	Deployment
0	70003	104205	903173.0
1	69943	104202	529979.0
100	69811	103402	561342.0
500	69604	103207	586867.0
500000	69598	103183	651665.0

TABLE V: Gas costs for deployment and for each function using the masking method on bytes32 utilizing Libraries.

Optimizer Runs	Register	CreateCharacter	Deployment
0	70003	67581	754613.0
1	69943	67414	508014.0
100	69811	66904	538621.0
500	69604	66835	556054.0
500000	69598	66835	569032.0

TABLE IV: Gas costs for deployment and for each function using the masking method on a uint.

Optimizer Runs	Register	CreateCharacter	Deployment
0	70003	66620	551800.0
1	69943	66365	378022.0
100	69811	65924	402120.0
500	69604	65855	419559.0
500000	69598	65855	432409.0

apparatuses can identify, making security studies and scientific categorizations fundamental for further developing agreement wellbeing [63], [33], [64].

XVII. METERING AND BILLING OF ENERGY ON ETHEREUM

This chapter explores the inefficiencies in the energy market and how blockchain can address them, particularly through smart contracts. It outlines the benefits of blockchain for energy applications, then presents the business logic for an energy metering and billing system implemented on Ethereum, incorporating methods for scalability and security.

A. Energy Market Inefficiencies

The energy market faces several inefficiencies, including:

- **Transaction complexity:** Increased complexity with more participants.

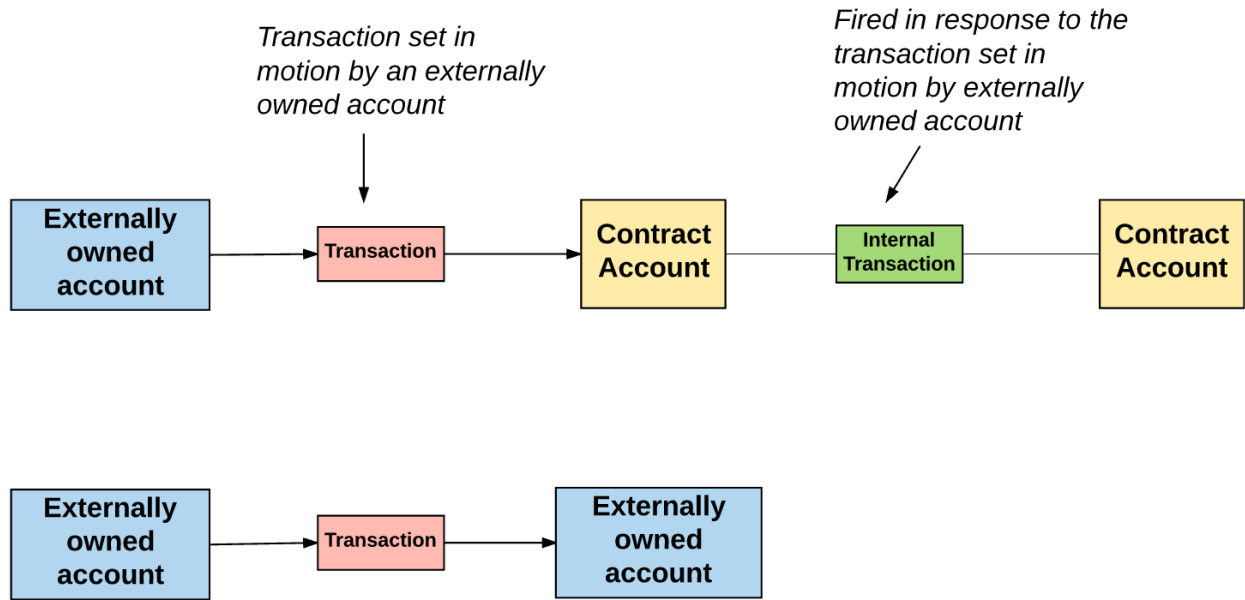


Fig. 7: Transaction made by an EOA to an EOA or a contract [18]

TABLE VI: Selected weights for evaluation criteria.

Name	Weight	Description
Readability	0.25	How readable is the code
Portability	0.2	How easy is it to use the code in another smart contract?
Gas Efficiency	0.3	How gas efficient is the code?
Security	0.25	How robust and secure is the code?

TABLE VII: Evaluation of each method

Name	Weight	M1	M1 (W)	M2	M2 (W)	M3	M3 (W)
Readability/Maintainability	0.25	5	1.25	3	0.75	4	1
Portability	0.2	5	0.8	3	0.6	4	0.8
Gas Efficiency	0.3	1	0.3	5	1.5	4	1.2
Security	0.25	5	1.25	4	1	4	1
Total	1	16	3.8	15	3.85	16	4

TABLE VIII: Slither is able to detect more vulnerabilities compared to other tools. Updated table from [64]

Vulnerability	Oyente	Remix	Securify	SmartCheck	Mythril	Slither
Reentrancy	Y	Y	Y	Y	Y	Y
Time-Order Dependency	Y	Y	Y	Y	Y	N
High Gas	N	Y	N	Y	Y	N
Unprotected Function	N	N	N	N	Y	Y
No Constructor	Y	N	N	N	Y	Y
tx.origin	N	Y	Y	Y	Y	Y
blockhash	N	Y	N	N	N	Y
Variable Shadowing	N	N	N	N	N	Y
Mapping in struct	N	N	N	N	N	Y

- **Predictability and Reliability:** DERs are less predictable than traditional resources.
- **Empowered prosumer:** Insufficient tools for prosumers to manage energy.
- **Geographic mismatches:** Inefficient energy transmission over long distances.
- **Trust and Security:** Ensuring trust and security for new participants.

B. Blockchain Use-Cases in the Energy Market

Blockchain enables solutions for several energy market challenges, such as:

- **Supply chain tracking and logistics:** Reduces fraud and ensures validity.
- **Energy metering and billing:** Timestamped records on the blockchain for accurate billing.
- **P2P energy trading:** Enables decentralized energy exchanges.
- **P2P energy micropayments:** Facilitates low-cost, real-

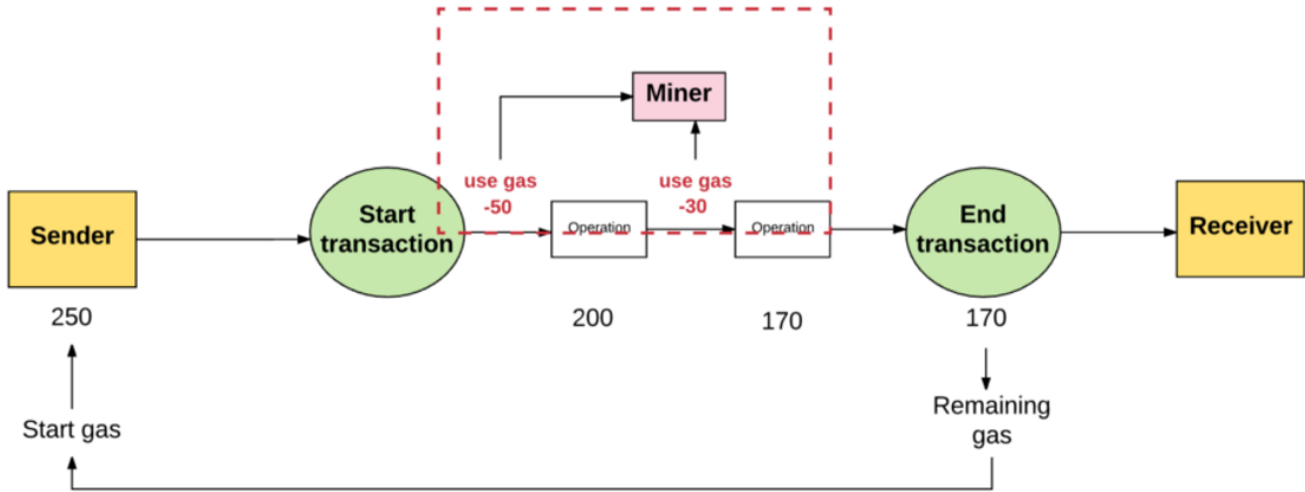


Fig. 8: Successful transaction [18]

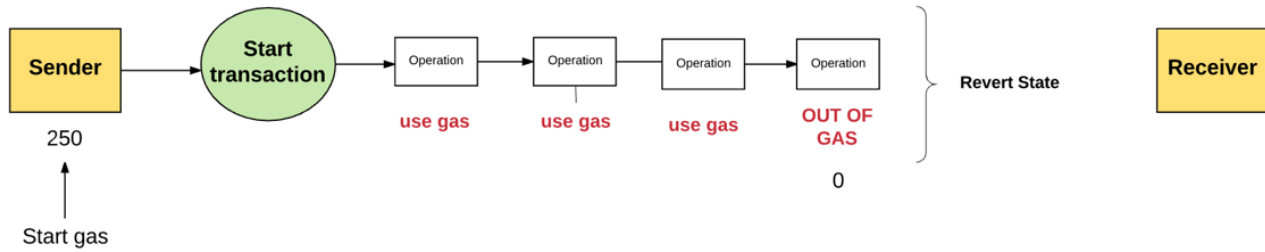


Fig. 9: Failed transaction that ran out of gas [18]

time payments for energy.

C. Business Logic

In collaboration with Honda R&D Germany, I developed a smart contract suite to track and bill energy consumption for the company's headquarters. The smart contracts are optimized for gas efficiency and robustness using best practices and security measures.

1) *Metering of Energy*: A **Smart Meter** logs energy readings to a monitoring server. These readings are then pushed to the blockchain. A **Virtual Meter** aggregates the readings of multiple smart meters across a building. Calculations for Virtual Meters are done off-chain for efficiency and then submitted to the blockchain.

2) *Accounting Logic*: Energy consumption is tracked through a hierarchy of **Business Units (BUs)** and **Departments**. Consumption is forwarded to delegates using headcount or fixed splits. Once a department's consumption is forwarded, its energy consumption is cleared.

D. Smart Contracts

The system uses several smart contracts to handle metering, billing, and access control. The key contracts are:

- **MeterDB**: Tracks energy consumption and timestamps for each meter.
- **AccountingDB**: Stores department consumption data and manages the forwarding of consumption to delegates.
- **Contract Registry**: Resolves contract names to addresses for easier management of contract interactions.
- **Access Control**: Ensures only authorized users can execute critical functions.

XVIII. CLIENT SIDE

As described in Section ??, smart contract functions get executed only in response to transactions made by EOAs⁵. Since the goal is to have an automated process of pulling data from and pushing data to the smart contracts, I implement

⁵Externally Owned Accounts, as described in Section ??

TABLE IX: Required variables and sizes to describe a Department.

Name	Type	Comment
active	bool	Department active flag
headcount	uint7	Headcount percentage, between 0 and 100
power	uint80	Department power from its virtual meters, up to 2**80
delegatedPower	uint80	Department power that has been delegated to it, up to 2**80
clearedPower	uint80	Department power that has been cleared, up to 2**80

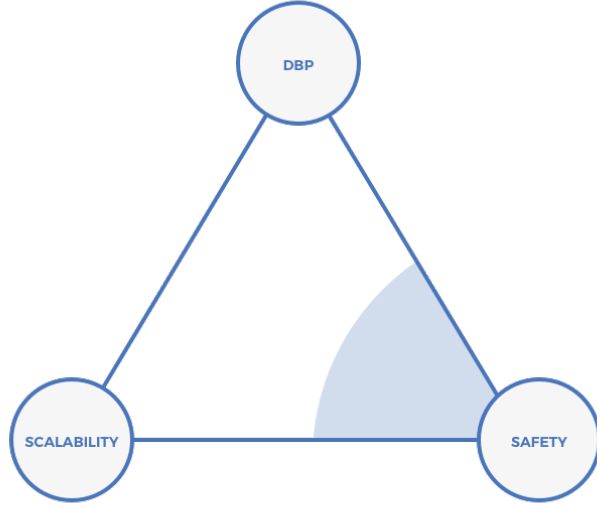


Fig. 10: The Scalability Triangle [53]

```

1 Character memory c;
2 c.playerID = uint16(playerID);
3 c.creationTime = uint32(creationTime);
4 c.class = uint8(class);
5 c.race = uint8(race);
6 c.strength = uint16(strength);
7 c.agility = uint16(agility);
8 c.wisdom = uint16(wisdom);
9 c.metadata = bytes18(metadata);

(a) Method 1: CreateCharacter

1 Character memory c = Characters[index];
2 return (
3   c.playerID,
4   c.creationTime,
5   c.class,
6   c.race,
7   c.strength,
8   c.agility,
9   c.wisdom,
10  c.metadata
11 );

```

Fig. 11: Method 1 API

client scripts which perform that functionality. I use Python and Web3.py to implement utilities and multiple command line interfaces (CLIs) which are used to interact with the smart contracts. A module is also developed for interacting with the REST APof the monitoring server. A separate keypair is generated for every role mentioned in Section ??, and each CLI is designed to function only with by using the corresponding role's private key.

```

1 uint c = uint256(playerID);
2 c |= creationTime << 16;
3 c |= class << 48;
4 c |= race << 52;
5 c |= strength << 56;
6 c |= agility << 72;
7 c |= wisdom << 88;
8 c |= metadata << 104;

(a) Method 2: CreateCharacter

1 return (
2   uint16(c),
3   uint32(c >> 16),
4   uint8((c >> 48) & uint256(2**4-1)),
5   uint8((c >> 52) & uint256(2**4-1)),
6   uint16(c >> 56),
7   uint16(c >> 72),
8   uint16(c >> 88),
9   bytes18(c >> 104)
10 );

```

Fig. 12: Method 2 API

TABLE X: Required variables and sizes to describe a Meter.

Name	Type	Comment
meterID	bytes8	A Meter's ID can be stored in 8 bytes
lastPower	uint48	Previous power reading, up to 2**48
lastTimestamp	uint32	Previous timestamp, up to 2**32
power	uint48	Current power reading, up to 2**48
timestamp	uint32	Current timestamp, up to 2**32

A. Utilities

Web3.py and Contract Wrappers: I created a wrapper around the web3 library in order to provide a generic interface when connecting to a node. That way, a web3 instance can be obtained by calling `getWeb3(endpoint)`, where `endpoint` is the Ethereum node IP and RPC port. Generally, in order to send a transaction, it must first be signed with the sender's private key. When a user hosts their own node, they can store their private key encrypted on the node and unlock the account when they want to send a transaction. In the case where multiple clients need to connect to a third-party node, there is a need to be able to send transactions that are already signed, without having to expose the private keys to the node.

I define a base class called `Contract` that provides functionality for interacting with a smart contract. It allows signing transactions locally and broadcasting them to an Ethereum node. It is able to track the nonce of an account which means that an account can submit multiple transactions without waiting for the previous one to get confirmed. During initialization, it receives the ABand address of the corresponding contract, along with a private key for signing the transactions. The exposed function `sign_and_send` is used to specify which function of the contract to call and submit it for execution to an Ethereum node. I use the `Contract` class throughout my implementation to create user-friendly interfaces for the functions of each contract while maximizing code reusability and security by signing transactions locally.

Monitoring Server and Rest API: As mentioned in Section ??, there is no direct access to each meter's reading, which

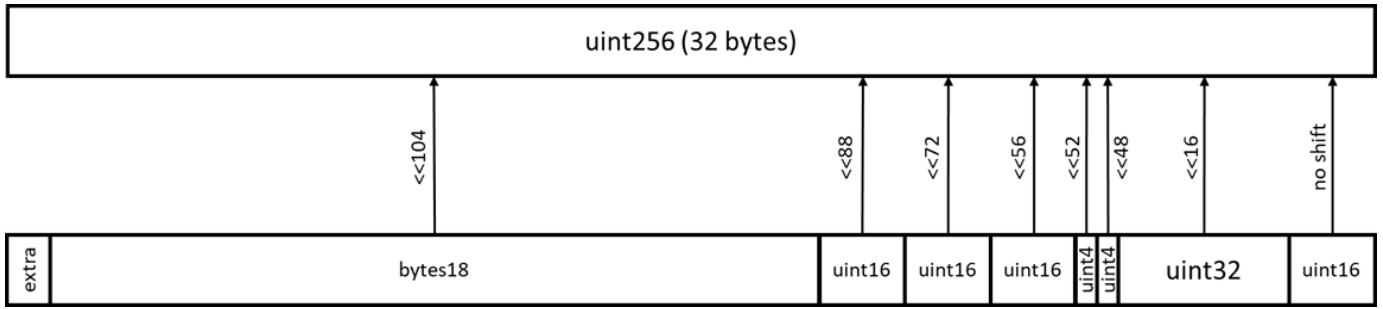


Fig. 13: Encoding data in a uint256 variable

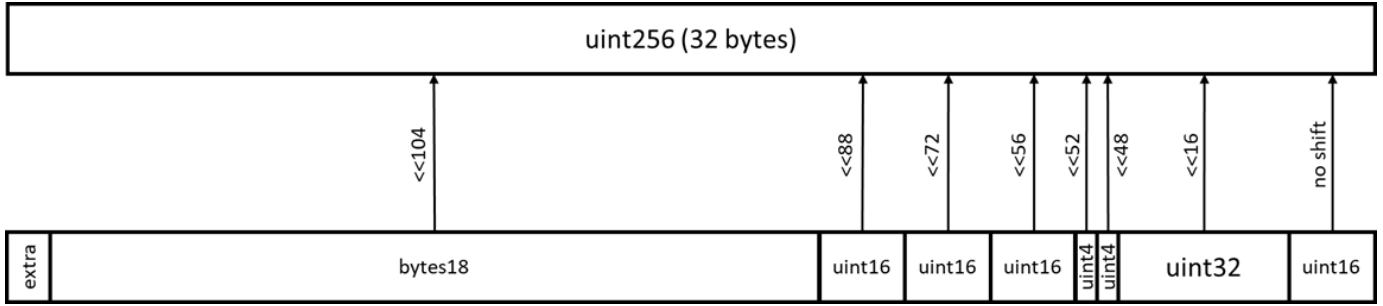


Fig. 14: Encoding data in a uint256 variable

```

1 bytes32 c = c.SetPlayerID(playerID);
2 c = c.SetCreationTime(creationTime);
3 c = c.SetClass(class);
4 c = c.SetRace(race);
5 c = c.SetStrength(strength);
6 c = c.SetAgility(agility);
7 c = c.SetWisdom(wisdom);
8 c = c.SetMetadata(metadata);

```

(a) Method 3: CreateCharacter

```

1 bytes32 c = Characters[index];
2 return (
3     c.GetPlayerID(),
4     c.GetCreationTime(),
5     c.GetClass(),
6     c.GetRace(),
7     c.GetStrength(),
8     c.GetAgility(),
9     c.GetWisdom(),
10    c.GetMetadata()
11 );

```

Fig. 15: Method 3 API

forces us to access all readings through a REST API. I implemented a module which gets instantiated with a Meter id as a parameter, and is able to interact with the monitoring server API. The primary function used is `get_single_reading` which fetches 1 reading from the monitoring server (by default the most recent one). Implementing the whole set of functionality provided from the AP was considered out of scope because it did not provide any additional information for my use-case.

B. Administrator CLI

The administrator role is used to grant or revoke role permissions to operators. The CLI interacts with the ACL smart contract which has been initialized as described in Section ?? . As input, it expects an action (grant, revoke) and a json configuration file with the address of each account and its

corresponding role to be granted or revoked. If no configuration file is supplied, it is expected that a role and address are provided as command line arguments. A user can additionally supply the `fund` parameter to send ether for the gas costs the operators will incur for interacting with their corresponding smart contracts.

C. Registry Operator CLI

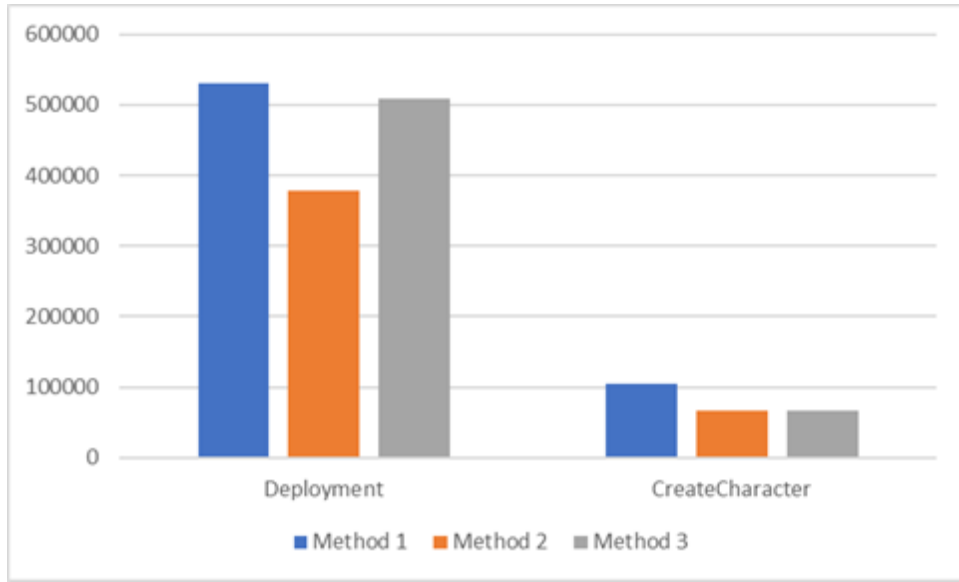
The registry operator's role is used to enable or disable a contract in the Registry contract. It can be utilized to register a new contract name and address at a later point in time or to remove an existing contract from the listing. The CLI also provides the functionality to resolve a contract name to its address or a contract address to its name.

D. Meter Operator CLI

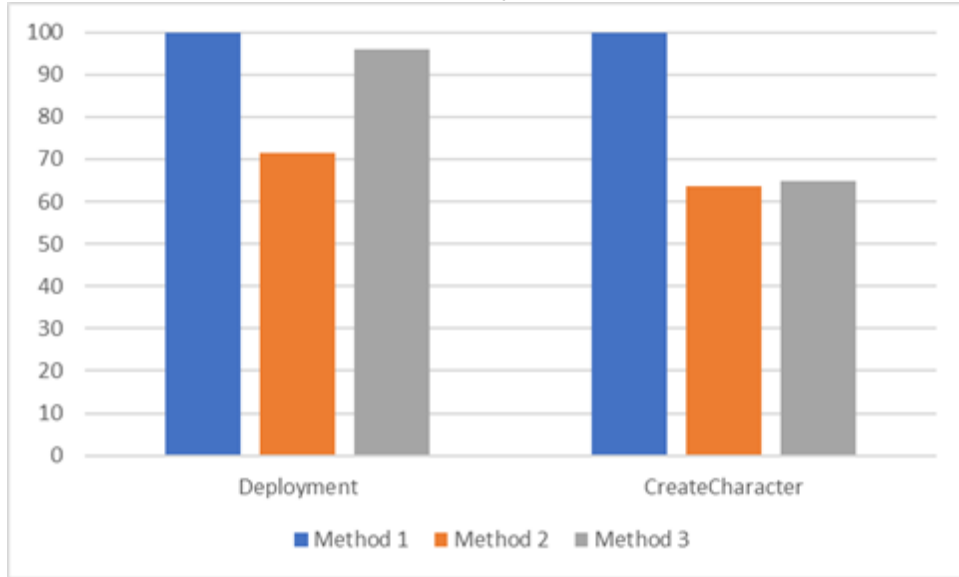
The meter operator role is used to activate or deactivate smart meters in the MeterDB contract. A meter can only call the `ping` function only if it is activated by the operator. The CLI expects as input a json configuration file with the address of each meter and its id. If no configuration file is supplied, it is expected that an address and a meter id are provided as command line arguments. If the `deactivate` parameter is given, the CLI tries to deactivate the meters in the list that are already activated. The operator can additionally supply the `fund` parameter to send ether for the gas costs the meters will incur for calling `ping`.

E. Accounting Operator CLI

The accounting operator role is used to activate or deactivate departments in the AccountingDB contract. After activation,



(a) Absolute values in gas for each method



(b) Percentage comparison between each method

Fig. 16: Gas cost comparison between the 3 proposed methods.

the operator sets each department's headcount, virtual meters, subdepartments and delegates according to the business logic supplied in a json configuration file that must be supplied as a command line argument.

F. Single and Multiple Meters CLI

A meter module is implemented which is responsible for connecting to MeterDB and the monitoring server APas explained in Section XVIII-A. The meter first needs to be activated by the meter operator as explained in Section XVIII-D. The client is responsible for polling the monitoring server and ping any new power readings to the smart contract. The single-meter CLI expects the meter's private key as input, and retrieves the meter id from the smart contract in order

to connect to the monitoring server. A frequency parameter can also be supplied to set how often a meter will poll the monitoring server for new data. This process runs in an infinite loop, or it can be modified to run as a cronjob.

Due to having multiple smart meters in my use-case, a multiple-meters CLI is also implemented which is responsible for launching multiple single-meter processes, each one running independently of each other. The output of each process is saved in a log file.

G. Single and Multiple Virtual Meters CLI

A virtualMeter module is implemented which is responsible for connecting to MeterDB and pulling the reading of its connected meters as described in Section ???. Along

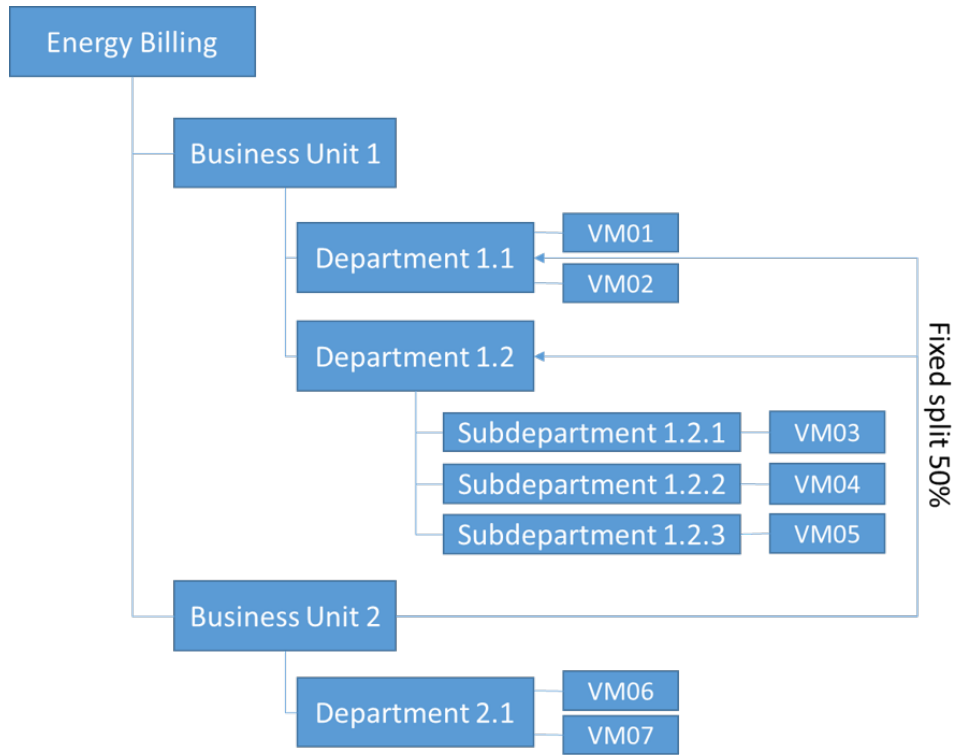


Fig. 17: Relationship between Departments and Delegates.



Fig. 18: The consumption of Virtual Meters gets calculated from the consumption of their associated Smart Meters.

with the virtual meter's private key, a function is supplied to the CLAs arguments which specifies how the Virtual Meter consumption is calculated from its associated meters. The virtual meter and the meters included in the formula must be previously activated by the meter operator. Given the formula, each referenced meter's readings are fetched, and the formula is evaluated⁶ before ping the final value to the smart contract. The process runs in an infinite loop, and can be modified to run as a cronjob.

Similarly to normal smart meters, a multiple-virtualmeters CLIs implemented

which is responsible for launching multiple single-virtualmeter processes, each one running independently of each other. The output of each process is saved in a log file.

H. Accountant CLI

The accountant module is responsible for pulling all the readings associated with a department, calculating the total amount of energy to be billed to the department as well as performing any forwarding of energy to a department's delegates. The accountant is simultaneously connecting to MeterDB and AccountingDB. Even though the accountant role is not authorized to access the functions related to ping

⁶Using the module: <https://github.com/Axiacore/py-expression-eval>

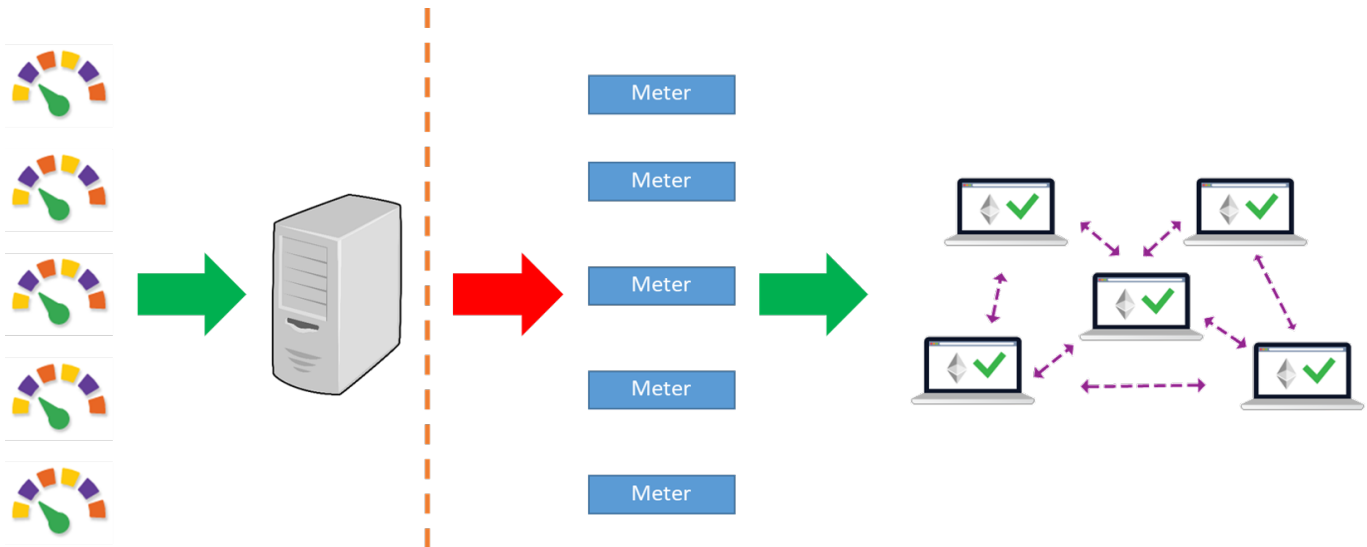


Fig. 19: Meter Data gets pushed to the blockchain after getting pulled from the monitoring server

or operating in `MeterDB`, accessing the data for each meter can be done without supplying the contract instance with a private key⁷

The departments are processed in order from the inner-most tiers towards the top level departments, as described in Section ???. The flowchart in Figure 21 shows the process that each department goes through in order to correctly calculate its final consumption.

I. Execution Pipeline

Having described how each module of the client side works, in Figure 22 I show the order in which the client side programs get executed to provide the desired metering and billing functionality. Firstly, the administrator grants all of the needed privileges to the corresponding operators' addresses. Then, the meter operator activates all meters, and all virtual meters. The accounting operator activates and initializes the accounting settings for each department. Each meter and virtual meter starts pushing their values asynchronously to the `MeterDB` contract, while at the same time the accountant is able to bill each department according to the logic which was previously specified by the accounting operator.

XIX. EVALUATION OF IMPLEMENTATION AND RESULTS

I have described an ecosystem of smart contracts that combined with their corresponding client scripts can perform metering and billing of energy with complex business logic. I proceed to evaluate the performance of the implementation described in Chapter ??, and explain how the findings from Chapter ?? and Chapter ?? are applied to enable better scalability and robustness in the contract design.

⁷Viewing data stored in a smart contract is free as it involves directly querying the node for its storage and does not involve any state mutating transaction. As a result, no private key is needed to access the meter readings.

XX. SCALABILITY

On scalability, I applied Method 3 (??) and followed the rules from Section XIV-A to save gas. Method 2 was not evaluated on the smart contracts as it would require significant refactoring, and based on the results from Chapter ?? it is expected to have similar performance. More specifically:

- 1) The optimizer flag was activated during compilation.
- 2) Using libraries via Method 3 for storing meter readings and department values resulted in significant code reuse, and reduction of gas costs. Table XI illustrates the gas savings that the proposed method provides, for each function of the contracts.
- 3) Any meter, department that is no longer used can be deactivated, freeing up storage in the smart contract.
- 4) Any iterations over arrays are initializing the break condition outside of the loop, reducing the number of `SLOAD` operations done and thus saving gas.
- 5) In all cases, `string` is avoided and `bytes32` is used.
- 6) The minimal amount of data needed is stored in the smart contract. The latest reading and timestamp values are needed for each meter and department, and each department additionally requires storing its hierarchical relationship with other departments, virtual meters and its delegates. The formula for each virtual meter (??) is stored locally because it would significantly increase gas costs to store an arbitrarily long `string` variable in the smart contract.
- 7) Events are emitted for each action providing an inexpensive way to store the historical data of a meter or department. Emitting an event requires significantly less gas compared to storing a value in contract storage [65]. The disadvantage is that event data cannot be accessed by another smart contract which is acceptable in this case as historical data are only meant to be accessed

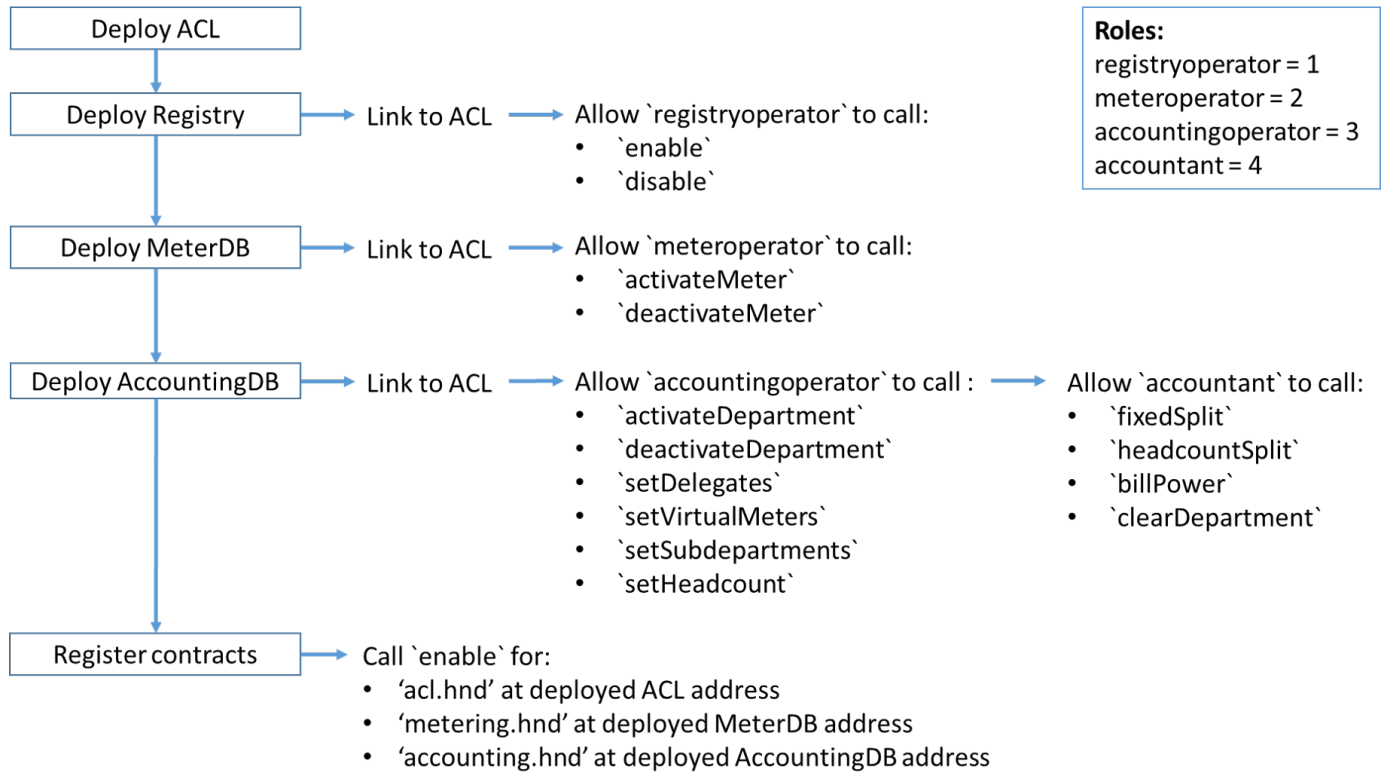


Fig. 20: The full deployment pipeline enforcing access control

directly by a client and not by a smart contract.

TABLE XI: Gas costs comparison between Method 1 and Method 3 for storing meter readings and department energy consumptions, optimizer runs = 200

Contract	Method 1	Method 3	Improvement
MeterDB::deployment	793279	637403	20%
AccountingDB::deployment	1691164	1088253	36%
MeterDB::ping	57844	31422	46%
AccountingDB::billPower	82025	28250	65%
AccountingDB::fixedSplit	98126	47260	51%
AccountingDB::headcountSplit	231305	55024	76%

XXI. SECURITY

Having studied and understood past literature as well as attacks on smart contracts, the implementation is developed so that none of the known attacks can be reproduced. In Section ?? I evaluated that Slither can detect a wide variety of vulnerabilities, and can outperform at most cases the other available security auditing tools. Given that the written code does not have

- 1) Reentrant functions since there is no external call to an address controlled by an attacker.
- 2) Functions dependent on block inclusion time.
- 3) block.hash
- 4) tx.origin
- 5) Mappings inside structs.

the only vulnerabilities to look for are 'Variable Shadowing' and 'No Constructor'. The only tool able to find both of the issues is Slither, which gave no positive results for the written

contracts. Mythril was also executed in order to detect 'No Constructor', which also gave negative results. These results, give us confidence regarding the robustness of the developed smart contracts against known attacks.

I utilized and applied an access control list to the implemented smart contracts to enforce role based access control. Roles are allowed to access only the functions of a smart contract that they absolutely must have access to, following the security principle of least privilege. Each user is then granted a role, which provides a portable method to revoke and modify the permissions of a user.

As smart contract ecosystems become more complex and the number of participating parties increase, ways to enforce access control are going to be increasingly important. my approach allows for a central directory of permissions to exist, which makes it efficient to modify the permissions of each actor on a per-contract basis.

XXII. COMPLETE OVERVIEW

The final implementation was tested with a setup of 48 smart meters, 44 virtual meters and 20 departments. The simulation was conducted on a Dell Laptop running an Ubuntu 16.04 LTS Virtual Machine. The Virtual Machine was given 4 GB of RAM and used 2 Intel i5-5300U CPUs, at 2.3Ghz with 2 cores each. Due to having to launch 94 Python processes the required RAM for the simulation was consistently in the range of 2.5 GB. When the meters were not pinging, the CPU usage did not exceed 5%. When the meters are interacting with

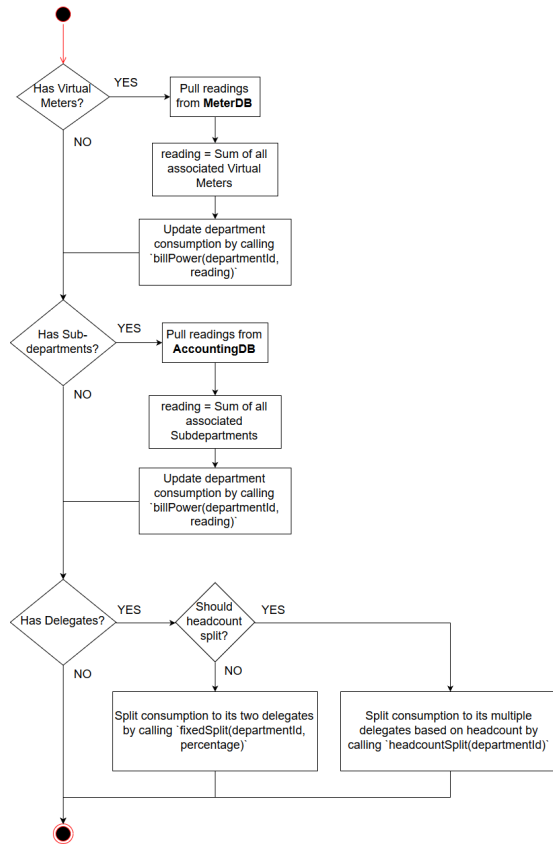


Fig. 21: Consumption calculation for a Department

the monitoring server a 15% spike in CPU usage momentarily occurs and when the meters ping their values to the blockchain CPU usage reaches approximately 50%. This is expected due to the large amount of processes, and it is safe to assume that a device such as a Raspberry Pi would not have any issue running up to 25 meter clients considering the CPU and RAM consumption required by each client. The deployment of the smart contracts was firstly tested on a local ganache⁸ testnet instance, then on a locally deployed testnet of 2 geth instances, where 1 was acting as a node and another as a miner, and finally on the Ropsten public testnet. Due to company requirements, there was no deployment to a public network, however since deployment to Ropsten was successful, it is expected that deployment to the mainnet will be successful as well.

Finally, I analyze the expected gas costs for operating the metering and billing logic of the developed smart contracts on the Ethereum mainnet for a year:

- 1) **Meters:** The monitoring server is updated every 12 minutes, however since the change in values is very small, I poll the monitoring server and update the values of the smart meters every 2 hours⁹. As a result, the

⁸Software which allows setting up a local private blockchain with funded accounts and instant block confirmation times, shown in Figure 23

⁹The selected frequency was a company requirement, as more frequent pings would increase cost and provide no significant advantage in granularity

operating cost for pinging the values of 48 meters for a year¹⁰ is: 48 meters * 31422 gas * 0.5 readings / hmy * 24 hours * 365 days * 1 Gwei/gas = 6.6 ether, which reduces to each meter needing 0.1375 ether in order to be able to interact with the smart contract at the specified frequency for a year.

- 2) **Virtual Meters:** Pushing the values of Virtual Meters is required to be done twice every month and so the total cost of running 44 virtual meters is: 44 virtual meters * 31422 gas * 2 readings / month * 12 months * 1 Gwei/gas = 0.03 ether, which reduces to each virtual meter needing 0.0007 ether to interact with the smart contract at the specified frequency for a year.
- 3) **Billing:** Calculating the consumption of each business unit and performing cost forwarding for internal accounting is required to be done once every month. It was simulated that a full billing cycle of pulling values from virtual meters, updating each department's consumption and delegating it according to the business logic costs 0.016 ether per billing round for 20 departments with varying amounts of subdepartments, virtual meters and delegates, which means 0.016 ether per month and 0.192 ether per year.

XXIII. CONCLUSION AND FUTURE WORK

XXIV. CONCLUSION

In this Diploma Thesis I explained the functionality and applications of blockchain technology. Executing complex operations and transactions at scale can result in significant costs realized as transaction fees and as a result blockchain scalability is considered to be a large issue. A blockchain should be viewed as a tool and not a panacea and should be selectively used where there is need for a replicated append-only database that operates without trust in a third party about the integrity, availability and transparency of data.

Ethereum is a blockchain platform which allows the creation of Decentralized Applications by allowing Turing complete programs to be executed inside its Ethereum Virtual Machine. Ethereum inherits the scalability issues of blockchains. The functionality of smart contracts is limited by the throughput of the network, and it is impossible to perform large amounts of transactions while maintaining reasonably low fees. I claim that solving scalability in Ethereum requires adjustments both at the network level and at the smart contract level. I briefly described the network level scalability solutions and enumerate programming rules and best practices for better contract level scalability, as well as propose a technique for more efficient data storage in smart contracts.

On security, although the attacks on the Ethereum protocol were limited and quickly remediated, smart contracts that are insecure are consistently being developed and exploited in the wild. I went over past smart contract hacks which have

¹⁰The chosen gas price is $1GWe = 10^{-9}ether$ since the operations are not time critical. When there is a need for fast confirmations the price can be set higher.

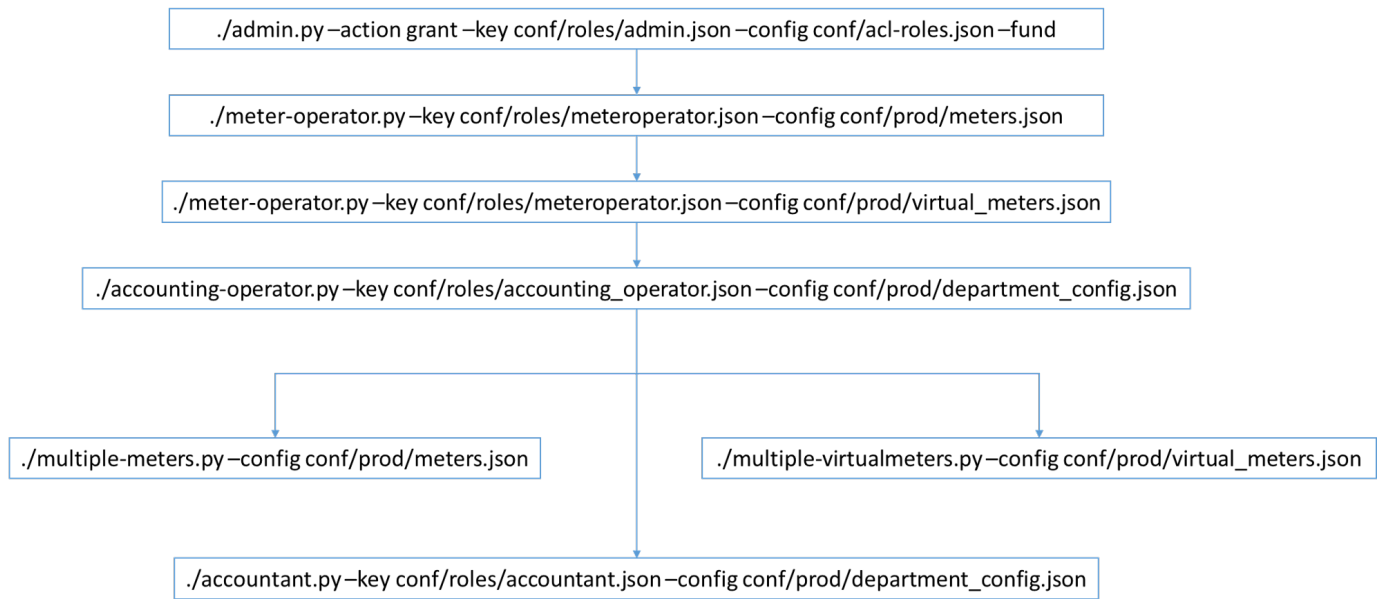


Fig. 22: The full execution process of metering and billing

led to heists of over 200 million dollars, as well as cases of funds being unspendable due to smart contract code that is no longer able to be run. I evaluated the usage of the tool Slither and found it to be superior to the other available open source auditing tools.

I developed a suite of smart contracts as well as client scripts that perform metering and billing of energy across a complex set of meters and departments based on a predefined accounting model set by the collaborating company, Honda R&D Europe Germany. The system was deployed on a public testnet and as a result is transparent and decentralized. The transactions are publicly inspectable and verifiable by anyone in the network. It is considered to be reasonably secure¹¹ given that no true positives were given from Slither and that the authoring of the code was made with taking the industry's best practices into mind, from Chapter ?? . An access control list was developed in Chapter ?? to only allow specific actors to have access to functions of the smart contracts. Finally, the system's scalability is improved by following the scalability rules and by utilizing the proposed authoring technique for storing data from Chapter ?? .

XXV. FUTURE WORK

All of the identified research topics for the Diploma Thesis were explored adequately.

Future work includes research on compiler optimization techniques to remove redundant operations and optimize parts of the reading and writing to storage, as a short term contract level scalability solution. Research on network level scalability should move towards Proof of Stake consensus algorithms to remove the need for the energy-consuming Proof of Work,

¹¹No system can be fully secure, unless formally verified.

as well as further research should be performed on the implementations of sidechains, state channels and sharding.

On security, smart contract languages must be developed which by default do not allow basic vulnerabilities such as TheDAO, and ones which are formally verifiable, in order to be able to mathematically prove that the code is not going to deviate from its intended results. Auditing tools should be further improved to analyze not only source code, but bytecode as well. Static analysis is not enough to detect vulnerabilities that are unknown, so there must be progress in the area of symbolic analysis tools.

Due to the aforementioned scalability issues, a use-case involving frequent transactions between multiple devices in an IoT scenario, such as my implementation, has large operational costs and it maybe the case that a permissioned blockchain can perform better, or an alternative distributed ledger architecture with high throughput and low transaction costs. A final improvement on my system is making it so that the client script for each meter is able to be run on the meter itself, thus removing the need for the monitoring server, which is a single point of failure.

APPENDIX A SMART CONTRACT DEVELOPMENT

In this Appendix I go over the steps a developer needs to go through in order to be able to code, test, debug and deploy DApps in Ethereum.

APPENDIX B SETTING UP A DEVELOPMENT ENVIRONMENT

In this section I explain the installation steps for the software needed for DApp development. Basic Linux knowledge is assumed. The described steps are tested in Ubuntu 17.04.

Firstly, an Ethereum client implementation must be installed on the system. I choose `geth`¹². I can download the latest version of `geth` from <https://github.com/ethereum/go-ethereum/releases>.

The tools that I will be using require NodeJS and Git. I install NodeJS via the Node Version Manager¹³ in order to have better management over the NodeJS versions.

I then install the tools and libraries used for having a proper development workflow. These tools are the Truffle Development Framework, Solium which is a linter to identify and fix style issues in Solidity, Web3.js which is the javascript library for interacting with the Ethereum ecosystem

Finally, I install Ganache which provides us with the ability to launch local testnet instances with prefunded accounts for local testing. Ganache has both a Graphical User Interface and a Command Line Interface version.

APPENDIX C

INTERACTING WITH ETHEREUM

By default, ganache starts listening at `localhost:8545` for incoming Ethereum JSON-RPC formatted messages¹⁴, and has 10 pre-funded accounts with 100 ether, as shown in Figure 23. In addition, any transaction that gets submitted gets instantly confirmed which allows for rapid debugging. I can attach to that and instance and interact with the testnet by using the command `geth attach http://localhost:8545`.

After being connected to a new testnet instance I can query the node for the balances of each account as well as make transactions as shown in Figure 24.

After a transaction gets submitted, its transaction hash is returned, and when it is mined in a block, its transaction receipt can be retrieved which contains additional information about that transaction as shown in 25. Similarly, the contents of a block can get fetched by querying `eth.getBlock(<number>)`.

APPENDIX D

DEVELOPMENT WORKFLOW

Truffle Framework and Ganache are the main tools used for smart contract development. When starting a new project, running the `truffle init` command results in the directory structure from Figure 26. Developed smart contracts are placed under the directory `contracts`, the deployment scripts under `migrations` and finally Mocha¹⁵ tests under `test`.

APPENDIX E

TOOL DEVELOPMENT

Having understood how `geth` is used for interacting with Ethereum and Truffle for developing, deploying and testing smart contracts, eventually I need to create my own tools. Web3 is the official library for interacting with Ethereum. I

have already installed it for javascript, however I also use the python version for scripting client side and because I were more comfortable with tools written in Python. `Web3.py` is installed through pip with `pip install web3`.

In case I want to interact with a public Ethereum network, I can either host my own local node and connect to `localhost:8545` like with ganache, or I can use a third-party public full node. Currently, the most reputable free provider for this service Infura¹⁶. Using Infura nodes requires first obtaining an APkey. After that, interacting with a node is a matter of initializing the web3 instance, as shown in Figure 27 for Javascript and in Figure 28 for Python.

APPENDIX F

SCALABILITY

APPENDIX G

ADVANTAGES OF THE OPTIMIZER

The Solidity compiler optimizer operates on assembly. It is able to analyze the compiled code and break it down in blocks, where each instruction gets analyzed [66]. It tries to simplify expressions which can be evaluated at runtime, and 'pack' items that are less than 32 bytes long into a single storage slot, according to the following rules:

- 1) The first item in a storage slot is stored lower-order aligned.
- 2) Elementary types use only that many bytes that are necessary to store them.
- 3) If an elementary type does not fit the remaining part of a storage slot, it is moved to the next storage slot.
- 4) Structs and array data always start a new slot and occupy whole slots (but items inside a struct or array are packed tightly according to these rules).

Variable packing is very efficient because writing to storage which has been already written to before by `SSTORE` requires only 5000 gas instead of 20000 (I). The execution cost for `set()` in Figure 29 is 60000¹⁷ because it initializes 3 storage slots, where it costs 45000 because it initializes 2 storage slots¹⁸. By using the optimizer, the third `SSTORE` is removed because the compiler can set both values with 1 `SSTORE` command in the same storage slot. As a result, the gas cost for calling `set()` with the optimizer is 40000 and requires only 2 `SSTORE` calls, where the optimizer is unable to provide any additional savings.

APPENDIX H

USING LIBRARIES IN SMART CONTRACTS

Libraries are contracts which cannot hold ether and do not have storage. Using libraries results in gas savings and code reuse. According to the Solidity Documentation, 'Libraries can be seen as implicit base contracts of the contracts that use them'. The functions of deployed Libraries are called by using

¹²<https://github.com/ethereum/go-ethereum>

¹³<https://github.com/creationix/nvm>

¹⁴<https://github.com/ethereum/wiki/wiki/JSON-RPC>

¹⁵<https://mochajs.org/>

¹⁶<https://infura.io/>

¹⁷The gas costs were obtained by calling the function in Remix IDE, <http://remix.ethereum.org/>

¹⁸The `uint128` items get packed in one storage slot.

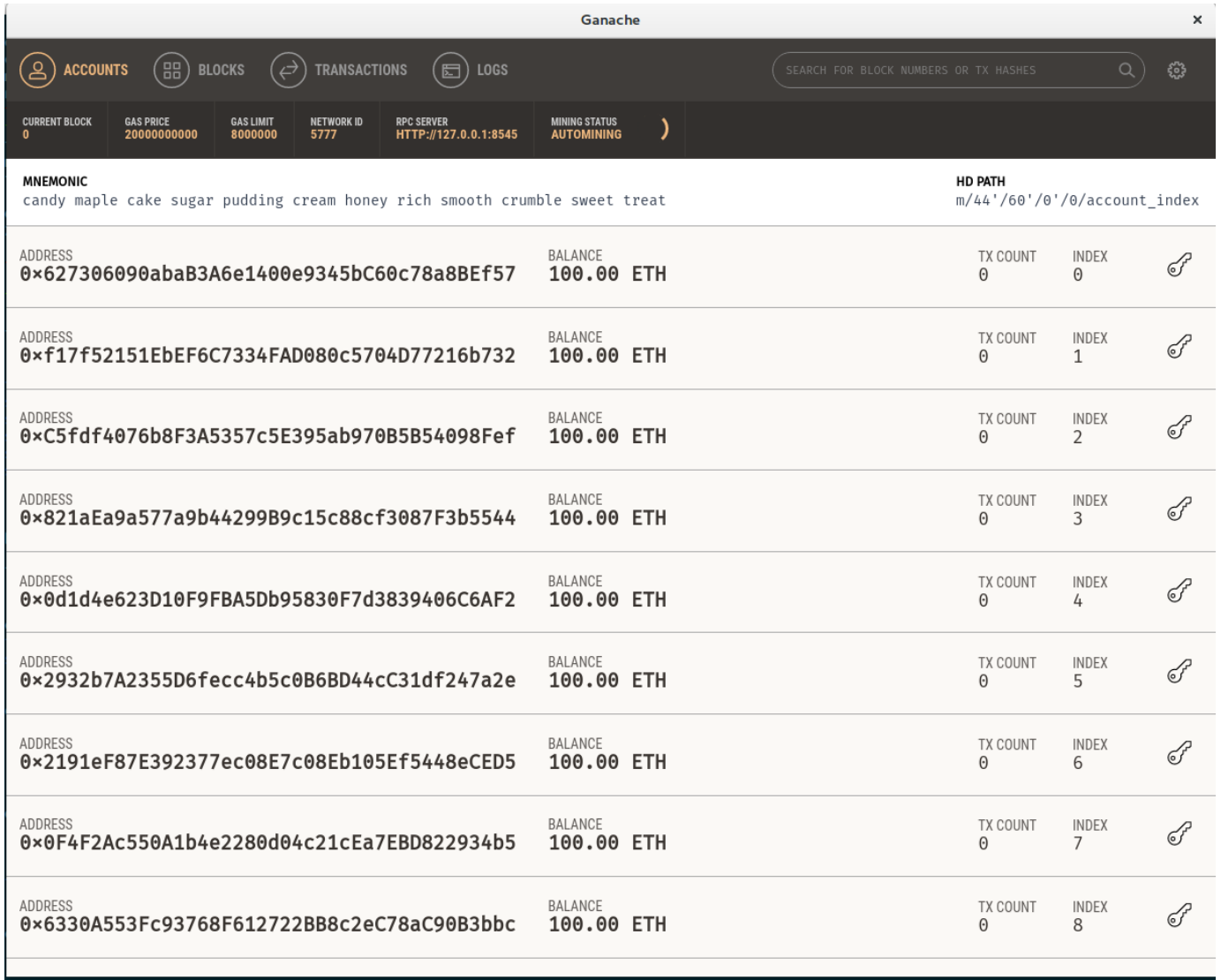


Fig. 23: Ganache User Interface

the opcode `DELEGATECALL`, which allows a function to be executed in the context of the caller. This results in contracts being able to delegate complex computation on an already deployed library's functionality, without having to deploy the library code. Libraries can be used to modify their associated contract's variables. Utilizing the `using` syntax, a function in a library can be called as a method of its first parameter as shown in Figure 30.

my implementation of the library for saving gas requires firstly defining the layout of variables in storage, as shown in Table XII for a Meter.

After that, I define a `mask` variable for each size of the datatypes I use and a variable that indicates how many shifts must be done to set or retrieve a variable. For example, setting `lastPower` requires shifting left 145 bits and applying a 48 bit mask. The exposed AP for setting the value is abstracted from the end user

TABLE XII: Layout of variables required to store a meter in a 256 bit word

Type	Name	Index	Bits
bool	active	0	1
bytes8	id	1	64
uint48	power	65	48
uint32	timestamp	113	32
uint48	lastPower	145	48
uint32	lastTimestamp	193	32

APPENDIX I COMPARING GAS COSTS ACROSS IMPLEMENTATIONS

In Section XIV-C I made gas comparisons between the 3 developed methods, in order to determine which was the most efficient in terms of gas. The process followed for each implementation was:

- 1) Compile the contract with N optimizer runs, where N in $[0, 1, 100, 500, 500000]$.

```
[~]: geth attach http://localhost:8545
Welcome to the Geth JavaScript console!

instance: EthereumJS TestRPC/v2.1.0/ethereum-js
coinbase: 0xe839dc48dcc914397e8edfcb783ea32e7e9d5cc
at block: 0 (Tue, 01 May 2018 11:05:21 PDT)
modules: eth:1.0 evm:1.0 net:1.0 personal:1.0 rpc:1.0 web3:1.0

> eth.blockNumber
0
> eth.getBalance(web3.eth.accounts[0])
1000000000000000000000000
> eth.getBalance(web3.eth.accounts[1])
1000000000000000000000000
> web3.eth.sendTransaction({
..... from: web3.eth.accounts[0],
..... to: web3.eth.accounts[1],
..... value: web3.toWei(1, 'ether')
..... })
"0x05dea26c053cf6d5e5f52e4f02b4be51e8239f38754435b8a99aabb6c208a791"
> eth.blockNumber
1
> eth.getBalance(web3.eth.accounts[0])
989999999999999999999999
> eth.getBalance(web3.eth.accounts[1])
1010000000000000000000000
```

Fig. 24: Querying a node and sending a transaction

- 2) Deploy the compiled ABand Bytecode to a local test-net¹⁹.
- 3) Get gas costs for deployment.
- 4) Execute each of the contract functions²⁰.
- 5) Get gas costs for each function call.

For that, I implemented a set of Python scripts responsible for automating the process. Specifically, I created:

- 1) A `getWeb3` script which allows connecting to an Ethereum node by providing an endpoint with the syntax `w3 = getWeb3(<endpoint>)`.
- 2) A `framework` script which exports a `Contract` class that can compile and deploy a smart contract when given its source code. It also provides a `call` method which allows calling a smart contract function and retrieve its gas usage.
- 3) A `comparison` script which utilizes the `framework` and the `pandas` library to create a dataframe with the gas costs for each function of the supplied smart contracts, for the specified optimizer runs.

¹⁹I used ganache as a testnet.

²⁰The functions were called once since the gas costs are deterministically the same for all calls

APPENDIX J SECURITY

APPENDIX K DELETING A STRUCT WITH MAPPING

Slither provides a module for detecting a vulnerability which involves deleting a `struct` which contains a mapping. I provide an example of a smart contract with this flaw in Figure 32. The steps to reproduce the vulnerability are the following:

- 1) Create a balance for a user by calling `createBalance(idx)`, let `idx` be 42.
- 2) Set the balance of a target address to a value, let the address be `0x1` and the value 100.
- 3) Delete the balance by calling `deleteBalance(42)`. It is expected that the `struct` at the index 42 will be deleted, along with all its contents.
- 4) Create a balance for another user by calling `createBalance(42)`. Retrieving that user's balance it can be seen that it is 100, instead of the expected 0.

APPENDIX L HONEYPOT SMART CONTRACTS

Since the second Parity bug no novel critical vulnerabilities have been identified in smart contracts. However, smart contracts that are architected to look vulnerable to known exploits started surfacing, with their true goal being to steal the funds of potential hackers. Hackers who attempt to exploit


```

1 pragma solidity ^0.4.21;
2
3 contract CannotPack {
4     uint128 a = 1;
5     uint256 b = 2;
6     uint128 c = 3;
7 }

```

(a) Variables cannot be packed

```

1 pragma solidity ^0.4.21;
2
3 contract CanPack {
4     uint128 a = 1;
5     uint128 b = 2;
6     uint256 c = 3;
7 }

```

(b) Variables can be packed

```

1 $ solc --asm CannotPack.sol | grep sstore | wc -l
2 3
3 $ solc --optimize --asm CannotPack.sol | grep sstore | wc -l
4 3
5
6 $ solc --asm CanPack.sol | grep sstore | wc -l
7 3
8 $ solc --optimize --asm CanPack.sol | grep sstore | wc -l
9 2

```

(c) 2 SSTOREs required for the CanPack contract compared to 3 for CannotPack

Fig. 29: The optimizer saves a call to SSTORE when it operates on packed storage variables.

```

1 library CounterLib {
2     struct Counter { uint i; }
3
4     function incremented(Counter storage self) returns (uint) {
5         return ++self.i;
6     }
7 }
8
9 contract CounterContract {
10     using CounterLib for CounterLib.Counter;
11
12     CounterLib.Counter counter;
13
14     function increment() returns (uint) {
15         return counter.incremented();
16     }
17 }

```

Fig. 30: Example of using the using X for Y syntax to create a Counter datatype [56]

KingOfTheHill contract. The `takeAll` function, is decorated by the `onlyOwner` modifier²² which looks for the value of `owner` in the `Owned` contract. As a result, the ‘real’ owner of the contract is unchanged, and the attacker is unable to claim ownership, effectively losing their funds. Running Slither on the contract can reveal the variable shadowing as shown in 35.

APPENDIX M IMPLEMENTATION

APPENDIX N GAS COSTS COMPARISON

In order to compare the gas consumption between implementations during testing I use `eth-gas-reporter`²³ which allows for to get gas usage per unit test, as well as retrieve average gas usage per method called. I configure `eth-gas-reporter` in the `truffle-config.js` file, and by prefixing a `truffle test` command with `GAS_REPORTER=1` I can

²²A modifier is a special function which gets executed before or after the function it *modifies*. They are primarily used to enforce access control in functions.

²³<https://github.com/cgewecke/eth-gas-reporter>

```

1 uint private constant mask48      = (1 << 48) - 1;
2 uint private constant _lastPower = 1 << 145;

```

(a) Mask and shift offsets for lastPower

```

1 function getProperty(bytes32 MeterData, uint mask, uint shift)
2     private pure returns (uint property) {
3     property = mask & (uint(MeterData) / shift);
4 }
5
6 function setProperty(bytes32 MeterData, uint mask, uint shift,
7     uint value) private pure returns (bytes32 updated) {
8     updated = bytes32(
9         (~(mask*shift) & uint(MeterData)) | ((value & mask) *
10             shift)
11     );
12 }

```

(b) Getting and setting a property

```

1 function setLastPower(bytes32 MeterData, uint48 power) internal
2     pure
3     returns (bytes32)
4 {
5     return setProperty(MeterData, mask48, _lastPower, power);
6 }
7
8 function getLastPower(bytes32 MeterData) internal
9     pure
10    returns (uint48)
11 {
12     return uint48(getProperty(MeterData, mask48, _lastPower));
13 }

```

Fig. 31: APfor setting and retrieving values by using a library

```

1 contract Bank {
2
3     struct BalancesStruct{
4         address owner;
5         mapping(address => uint) balances;
6     }
7
8     mapping(uint => BalancesStruct) stackBalance;
9     function createBalance(uint idx){
10         require(stackBalance[idx].owner == 0);
11         stackBalance[idx] = BalancesStruct(msg.sender);
12     }
13
14     function deleteBalance(uint idx){
15         require(stackBalance[idx].owner == msg.sender);
16         delete stackBalance[idx];
17     }
18
19     function setBalance(uint idx, address addr, uint val){
20         require(stackBalance[idx].owner == msg.sender);
21
22         stackBalance[idx].balances[addr] = val;
23     }
24
25     function getBalance(uint idx, address addr) returns(uint){
26         return stackBalance[idx].balances[addr];
27     }
28
29 }

```

Fig. 32: Mapping inside a struct

get enhanced tests as shown in Figure 36. The metrics shown in Table XI are obtained after running the 4 tests in the supplied `smartcontract-gascomparison` repository.

APPENDIX O ACCESS CONTROL

Modifiers in Solidity are used to change the behavior of functions, similarly to decorators in Python²⁴. The modifier in the case of Figure 3738 gets called before the call to `foo()`, and after the `require` statement gets verified, execution resumes from ‘_’.

The usage of modifiers has been particularly widespread on enforcing access control to create contracts where only specific addresses can access selected privileged functions of a contract.

In the naive case, if there were a number of smart contracts deployed by the same entity which had privileged functions

²⁴<https://www.python.org/dev/peps/pep-0318/>

APPENDIX P LISTENING FOR EVENTS

```

1 pragma solidity ^0.4.11;
2
3 contract Owned {
4     address owner;    function Owned() {
5         owner = msg.sender;
6     }
7     modifier onlyOwner{
8         if (msg.sender != owner)
9             revert();
10    }
11 }
12
13 contract KingOfTheHill is Owned {
14     address public owner;
15     uint public jackpot;
16     uint public withdrawDelay;
17
18     function() public payable {
19         // transfer contract ownership if player pay more than
20         // current jackpot
21         if (msg.value > jackpot) {
22             owner = msg.sender;
23             withdrawDelay = block.timestamp + 5 days;
24         }
25         jackpot+=msg.value;
26     }
27
28     function takeAll() public onlyOwner {
29         require(block.timestamp >= withdrawDelay);
30         msg.sender.transfer(this.balance);
31         jackpot=0;
32     }
33 }

```

Fig. 33: A honeypot which takes advantage of Variable Shadowing.

TxHash	Block	Age	From	To	Value	Contract
0x11713ba0712b71	4755038	116 days 19 hrs ago	0x00a07276a00e	0x46b76c55914b3	0.42 Ether	0x00000000
0x06e07c36a00e011	4755081	116 days 19 hrs ago	0x00a07276a00e	0x46b76c55914b3	0.42 Ether	0x00000000
0x571672029326	4748448	117 days 20 hrs ago	0x229c138701296	0x46b76c55914b3	0.21 Ether	0x00000000
0x455916434a232	4527953	154 days 20 hrs ago	0x1258501c5c084	0x46b76c55914b3	0.100000000001 Ether	0x00000000
0x53a0b0a467697	4525157	155 days 4 hrs ago	0x002028901c589	0x46b76c55914b3	0.1 Ether	0x00000000
0x08030a0b6762	4525139	155 days 4 hrs ago	0x002028901c589	Contract Creation	0 Ether	0x00000000

Fig. 34: Transactions trying to get ownership of the honeypot contract

callable only by an owner, the same pattern would have to be used in each contract. If a developer wanted to make the ownership check more flexible, they would have to modify the privileged members in each contract. This does not scale well, due to the number of transactions involved. In addition, if there were multiple roles, managing them on different contracts can become inefficient and has increased room for error.

I use the implementation of an Access Control List contract from DAppHub²⁵. This pattern involves deploying two contracts, where one gets inherited by any contract that wishes to have access control, and the other stores all the access control list logic.

DSRoles.sol allows the creation of up to 256 roles, where each role can be granted permission to call the function of a contract.

In order for a contract to use the ACL (authority), it needs to be linked with it by the owner by inheriting DSAuth.sol and calling the setAuthority function. Afterwards, any function that gets modified with auth will ask for permission from the linked instance of DSRoles.sol as shown in Figure 39. This allows for a modular approach where an administrator²⁶ can grant and revoke permissions from users, depending on their clearance level.

²⁵<https://github.com/dapphub/ds-roles>, <https://github.com/dapphub/ds-auth>

²⁶The version I use contains a centralized entity which handles permission. This can be re-implemented depending on the desired governance model

Events are an inexpensive way to store data in the Ethereum blockchain. An Event is a special kind of datatype that gets logged in the transaction receipt after a transaction gets mined. As a result, if I call a function that emits an event, that event is stored permanently in the blockchain for approximately 2000 gas. Listening for events is a fundamental process that web applications do in order to interact with a smart contract. I can also fetch all past events that a smart contract has emitted, which allows us to have a full historical log over some action that has happened in a smart contract.

For my implementation I implemented a proof of concept script written in Javascript which can fetch all the readings that have been pinged in the smart contract, and additionally filter them by Meter Id. The same script can be implemented in Python, however there are ongoing issues with it when interacting with ganache in web3.py²⁷. An example is shown in Figure 40 and the implementation is in Figure 41.

REFERENCES

- [1] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008.
- [2] V. Buterin *et al.*, "A next-generation smart contract and decentralized application platform," *white paper*, 2014.
- [3] N. Szabo, "Smart contracts: Building blocks for digital markets," 1995.
- [4] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum Project Yellow Paper*, vol. 151, 2014.
- [5] P. Rogaway and T. Shrimpton, "Cryptographic hash-function basics: Definitions, implications, and separations for preimage resistance, second-preimage resistance, and collision resistance," 2004.
- [6] Ethereum, "Ethereum," <https://github.com/ethereum/wiki/wiki/Ethereum>.
- [7] "Which cryptographic hash function does ethereum use?" <https://ethereum.stackexchange.com/a/554>.
- [8] "Monero," <https://getmonero.org/>.
- [9] "Zcash," <https://z.cash/>.
- [10] V. Buterin, "On public and private blockchains," <https://blog.ethereum.org/2015/08/07/on-public-and-private-blockchains>, 2015.
- [11] J. Morgan, "A permissioned implementation of ethereum supporting data privacy," <https://www.jpmorgan.com/country/DE/en/Quorum>.
- [12] "Hyperledger," <https://www.hyperledger.org/>.
- [13] "R3," <https://www.r3.com/>.
- [14] T. T., "Ethereum virtual machine illustrated," http://takenobu-hs.github.io/downloads/ethereum_evm_illustrated.pdf.
- [15] C. Dwork and M. Naor, "Pricing via processing or combatting junk mail," in *Annual International Cryptology Conference*. Springer, 1992, pp. 139–147.
- [16] Y. Sompolinsky and A. Zohar, "Secure high-rate transaction processing in bitcoin," 12 2013. [Online]. Available: <https://eprint.iacr.org/2013/881.pdf>
- [17] A. Gervais, G. O. Karame, K. Wüst, V. Glykantzis, H. Ritzdorf, and S. Capkun, "On the security and performance of proof of work blockchains," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '16. New York, NY, USA: ACM, 2016, pp. 3–16. [Online]. Available: <http://doi.acm.org/10.1145/2976749.2978341>
- [18] P. Kasireddy, "How does ethereum work, anyway?" <https://medium.com/@preethikasireddy/how-does-ethereum-work-anyway-22d1df506369>, 2017.
- [19] E. Hilbom and T. Tobias, "Applications of smart-contracts and smart-property utilizing blockchains," 2016.
- [20] "Bamboo: a morphing smart contract language," <https://github.com/pirapira/bamboo/>.
- [21] F. Schrans, S. Eisenbach, and S. Drossopoulou, "Writing safe smart contracts in flint," *Unknown*, 2018.

²⁷<https://github.com/trufflesuite/ganache-cli/issues/494>


```
[~/Diploma/security/tools/slither, master]: ./slither.py ../../honeypots/known-honeypots/shadowing-state-variable/KingOfTheHill.sol --disable-solc-warnings
INFO:Slither:Missing constructor in ../../honeypots/known-honeypots/shadowing-state-variable/KingOfTheHill.sol, Contract: KingOfTheHill (owner)
INFO:Slither:Shadowing in ../../honeypots/known-honeypots/shadowing-state-variable/KingOfTheHill.sol, Contract: KingOfTheHill, Contract/Vars: {u'owned': set([u'owner'])}
```

Fig. 35: Slither is able to detect the variable shadowing in owner

```
$ GAS_REPORTER=1 truffle test test/testMeterStructs.js

Using network 'test'.

Contract: MeterDB.sol
  ✓ Activates a meter and pings a value (1043241 gas)

-----
|                                     | Block limit: 6721975 gas | | | | | |
|---|---|---|---|---|---|---|
|                                     | 3 gwei/gas              | 681.61 usd/eth          |
|-----|-----|-----|-----|-----|-----|
| Contract | Method | Min | Max | Avg | # calls | usd (avg) |
|-----|-----|-----|-----|-----|-----|
| MeterDBStructs | activateMeter | - | - | 134274 | 1 | 0.27 |
| MeterDBStructs | activateMeterBatch | - | - | - | 0 | - |
| MeterDBStructs | deactivateMeter | - | - | - | 0 | - |
| MeterDBStructs | ping | - | - | 57844 | 2 | 0.12 |
|-----|-----|-----|-----|-----|-----|
| Deployments | % of limit |
|-----|-----|-----|-----|-----|
| MeterDBStructs | - | - | 793279 | 11.8 % | 1.62 |
|-----|-----|-----|-----|-----|

1 passing (8s)
```

Fig. 36: Running a truffle test with eth-gas-reporter

```
1 contract Owned {
2   address owner;
3   function Owned() public { owner = msg.sender; }
4   modifier onlyOwner {
5     require(msg.sender == owner);
6   }
7 }
8
9
10 contract Test is Owned {
11   function foo() onlyOwner {
12     /* do something only owner can do */
13   }
14 }
```

Fig. 37: Using the onlyOwner modifier to enforce access control for a function

```
1 function canCall(address caller, address code, bytes4 sig)
2   public
3   view
4   returns (bool)
5 {
6   if( isUserRoot(caller) || isCapabilityPublic(code, sig) ) {
7     return true;
8   } else {
9     bytes32 has_roles = getUserRoles(caller);
10    bytes32 needs_one_of = getCapabilityRoles(code, sig);
11    return bytes32(0) != has_roles & needs_one_of;
12  }
13 }
```

Fig. 38: canCall in DSRoles.sol checks if the sender is authorized to perform the function call according to the permissions set by the ACL

- [22] “Stateless smart contracts,” <https://medium.com/@childsmidment/stateless-smart-contracts-21830b0cd1b6>, 2017.
- [23] J. Poon and T. Dryja, “The bitcoin lightning network: Scalable off-chain instant payments,” <http://lightning.network/lightning-network-paper.pdf>, 2016.
- [24] “Raiden network,” <https://raiden.network/>.
- [25] “Funfair technologies,” <https://funfair.io>.
- [26] “Raiden network,” <https://counterfactual.com/>.
- [27] A. Back, M. Corallo, L. Dashjr, M. Friedenbach, G. Maxwell, A. Miller, A. Poelstra, J. Timón, and P. Wuille, “Enabling blockchain innovations with pegged sidechains,” URL: <http://www.opensciencereview.com/papers/123/enablingblockchain-innovations-with-pegged-sidechains>, 2014.
- [28] “Loom network,” <https://loomx.io>.
- [29] V. Buterin, K. Floersch, and D. Robinson, “Plasma cash: Plasma with much less per-user data checking,” 2018.

- [30] J. Poon and V. Buterin, “Plasma: Scalable autonomous smart contracts,” 2017.
- [31] M. Vukolić, “Rethinking permissioned blockchains,” in *Proceedings of the ACM Workshop on Blockchain, Cryptocurrencies and Contracts*, ser. BCC ’17. New York, NY, USA: ACM, 2017, pp. 3–7. [Online]. Available: <http://doi.acm.org/10.1145/3055518.3055526>
- [32] T. Chen, X. Li, X. Luo, and X. Zhang, “Under-optimized smart contracts devour your money,” *CoRR*, vol. abs/1703.03994, 2017. [Online]. Available: <http://arxiv.org/abs/1703.03994>
- [33] N. Atzei, M. Bartoletti, and T. Cimoli, “A survey of attacks on ethereum smart contracts sok,” in *Proceedings of the 6th International Conference on Principles of Security and Trust - Volume 10204*. New York, NY, USA: Springer-Verlag New York, Inc., 2017, pp. 164–186. [Online]. Available: https://doi.org/10.1007/978-3-662-54455-6_8
- [34] “An in-depth look at the parity multisig bug,” <http://hackingdistributed.com>.

```

1 modifier auth {
2     require(isAuthorized(msg.sender, msg.sig));
3     _;
4 }
5
6 function isAuthorized(address src, bytes4 sig) public view returns
7     (bool) {
8     if (src == address(this)) {
9         return true;
10    } else if (src == owner) {
11        return true;
12    } else if (authority == DSAuthority(0)) {
13        return false;
14    } else {
15        return authority.canCall(src, this, sig);
16    }
17 }

```

Fig. 39: Using the auth modifier to request permission from the linked authority (ACL)

```

1 $ ./get_meter_readings --rpc http://localhost:8545 --address "0
  xb549d25374337a3d2cc7bad7a4009d8c1c3f537" --meter "ELT01 ELT02"
2 Meter ELT01: 10 kWh at 1525281846
3 Meter ELT01: 20 kWh at 1525285175
4 Meter ELT02: 15 kWh at 1525281852
5 Meter ELT02: 22 kWh at 1525285166

```

Fig. 40: Retrieving all the historical readings for meters ELT01 and ELT02

com/2017/07/22/deep-dive-parity-bug/.

- [35] “A postmortem on the parity multi-sig library self-destruct,” <http://paritytech.io/a-postmortem-on-the-parity-multi-sig-library-self-destruct/>.
- [36] “Analysis of the dao exploit,” <http://hackingdistributed.com/2016/06/18/analysis-of-the-dao-exploit/>, 2016.
- [37] L. Luu, D.-H. Chu, H. Olickel, P. Saxena, and A. Hobor, “Making smart contracts smarter,” in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’16. New York, NY, USA: ACM, 2016, pp. 254–269. [Online]. Available: <http://doi.acm.org/10.1145/2976749.2978309>
- [38] ConsenSys, “Mythril,” <https://github.com/ConsenSys/mythril>.
- [39] “Echidna, ethereum fuzz testing framework,” <https://github.com/trailofbits/echidna>.
- [40] “Smartcheck,” <https://tool.smartdec.net/>.
- [41] “Securify,” <https://securify.ch/>.
- [42] “Zeus: Analyzing safety of smart contracts.”
- [43] K. Christidis and M. Devetsikiotis, “Blockchains and smart contracts for the internet of things,” *IEEE Access*, vol. 4, pp. 2292–2303, 2016.
- [44] A. Ahmad, “Integration of iot devices via a blockchain-based decentralized application,” 2017.
- [45] M. Thakur, “Authentication, authorization and accounting with ethereum,” 2017.
- [46] “Grid+,” <https://gridplus.io/>.
- [47] “Powerledger,” <https://powerledger.io/>.
- [48] “Brooklyn microgrid,” <https://brooklyn.energy>.
- [49] E. Mengelkamp, B. Notheisen, C. Beer, D. Dauer, and C. Weinhardt, “A blockchain-based smart grid: towards sustainable local energy markets,” *Computer Science - R&D*, vol. 33, no. 1-2, pp. 207–214, 2018. [Online]. Available: <https://doi.org/10.1007/s00450-017-0360-9>
- [50] “Mining,” <https://github.com/ethereum/wiki/wiki/Mining>.
- [51] “Sharding faq,” <https://github.com/ethereum/wiki/wiki/Sharding-FAQ>.
- [52] V. Buterin and G. Virgil, “Casper the friendly finality gadget,” 2017.
- [53] K. Samani, “Models for scaling trustless computation,” <https://multico.in.capital/2018/02/23/models-scaling-trustless-computation/>.
- [54] “Solc.acomeyinoneact,” <https://github.com/figs999/Ethereum/blob/master/Solc.aComedyInOneAct>.
- [55] “Gas costs from yellow paper – eip-150 revision (1e18248 - 2017-04-12),” https://docs.google.com/spreadsheets/d/In6mRqkBz3iWcOlRem_mO09GtSKEKrAsfO7Frgx18pNU/edit#gid=0, 2017.
- [56] J. Izquierdo, “Library driven development in solidity,” <https://blog.aragon.one/library-driven-development-in-solidity-2bebcdf88736>, 2017.
- [57] H. Jameson, “Faq: Upcoming ethereum hard fork,” <https://blog.ethereum.org/2016/10/18/faq-upcoming-ethereum-hard-fork/>, 2016.

```

1 #!/usr/bin/env node
2 const Web3 = require('web3')
3 const args = require('optimist').string("address").argv;
4 const truffleContract = require("truffle-contract");
5 const web3 = new Web3(new Web3.providers.HttpProvider(args.rpc));
6
7 function toAscii(input) {
8     return web3.toAscii(input).replace(/\u0000/g, '');
9 }
10
11 let conf;
12 if (args.abi) {
13     conf = require(args.abi);
14 } else {
15     conf = require('./conf/abi/MeterDB.json');
16 }
17
18 let address = args.address;
19 let abi = conf.abi;
20
21 const contract = truffleContract({ abi });
22 contract.setProvider(web3.currentProvider);
23 meterdb = contract.at(address);
24
25 let meters = {};
26 if (args.meter) {
27     meters = { "meterId": args.meter.split(" ") };
28 }
29
30 meterdb.Pinged(meters, { fromBlock: 0, toBlock: 'latest' }).get((
31     err, res) => {
32     if (!err) {
33         res.forEach(log => {
34             // event Pinged(address indexed meterAddress, bytes8
35             // indexed meterId, uint80 power, uint80 timestamp);
36             arg = log.args;
37             meterId = arg.meterId;
38             power = arg.power;
39             timestamp = arg.timestamp;
40             console.log('Meter ' + toAscii(meterId) + ': ' + power
41                 + ' kWh at ' + timestamp);
42         });
43     }
44 });

```

Fig. 41: Script to retrieve all meter readings

- [58] Y. Marcus, E. Heilman, and S. Goldberg, “Low-resource eclipse attacks on ethereum’s peer-to-peer network,” *Cryptology ePrint Archive*, Report 2018/236, 2018, <https://eprint.iacr.org/2018/236>.
- [59] “Ether thief remains mystery after \$55 million heist,” <https://www.bloomberg.com/features/2017-the-ether-thief/>, 2017.
- [60] “Parity multisig wallet exploit hits swarm city funds - statement by the swarm city core team.”
- [61] “Standardized ethereum recovery proposals,” [url-https://github.com/ethereum/EIPs/pull/867](https://github.com/ethereum/EIPs/pull/867).
- [62] “Eip-999: Restore contract code at 0x863df6bfa4,” <https://ethereum-magicians.org/t/eip-999-restore-contract-code-at-0x863df6bfa4/130>.
- [63] M. Alharby and A. van Moorsel, “Blockchain-based smart contracts: A systematic mapping study,” *CoRR*, vol. abs/1710.06372, 2017. [Online]. Available: <http://arxiv.org/abs/1710.06372>
- [64] A. Dika, “Ethereum smart contracts: Security vulnerabilities and security tools,” 2017.
- [65] “Technical introduction to events and logs in ethereum,” <https://media.consensys.net/technical-introduction-to-events-and-logs-in-ethereum-a074d65dd61e>.
- [66] “Solidity documentation,” <http://solidity.readthedocs.io/en/v0.4.21/miscellaneous.html>.